

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a52b8c159fee4afd9.jsonl`  
- SHA-256: `6898da235ef751f222c10cd5308f3ffc92362ae32762ee3c0abf495747315d0f`  
- Source modified: `2026-07-06T20:45:08+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T20:40:57.005Z)

Write a self-contained Python script `exp\_t2\_full.py` that runs FULL WASB shuttle-detection inference (heatmap -> postprocess -> OnlineTracker -> trajectory CSV) under torch 2.x, with two interchangeable front-ends (CPU cv2 vs GPU-resident PyNvVideoCodec), producing two trajectory CSVs so we can score them with the repo's eval harness. It runs on a Linux L4 VM (NOT here). Output ONLY the final script in a ``python block plus a short notes section. Do not run anything.

Read these local sources to replicate the EXACT WASB inference glue

```
- The reference implementation (follow its structure closely):  
`C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py` ? how it calls WASB's `build_detector(cfg)`,  
`detector.run_tensor(imgs, trans)`, accumulates per-frame detections (`det_by_fid`), builds the  
affine `trans`, calls `build_tracker(cfg)` / the OnlineTracker `refresh()`+`update()`, and  
writes the `Frame,Visibility,X,Y` trajectory CSV. Also `native_wasb_runner.py` in the same dir.  
- WASB's own code (local clone): `C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src` ?  
`detectors/detector.py` (build_detector, run_tensor, heatmap decode),  
`detectors/postprocessor.py` (sigmoid + blob/threshold decode), the tracker module (find it:  
`trackers/` or similar ? the OnlineTracker with refresh/update), `models/hrnet.py`,  
`configs/model/wasb.yaml`, `configs/dataset/badminton.yaml`, and how  
`get_affine_transform`/`trans` is built (`dataloaders/` or `utils/`).
```

A prior spike already proved the HRNet builds under torch 2.6 via
`models.hrnet.HRNet(OmegaConf.create(cfg))` with weights `torch.load(path, weights\_only=False)["model\_state\_dict"]`, input 512x288, 9-channel (3 RGB frames), ImageNet-normalize (/255 then mean=[0.485,0.456,0.406] std=[0.229,0.224,0.225]), sigmoid on the heatmap. Prefer building the model directly (avoid Hydra runtime), but USE WASB's own postprocessor + tracker classes (import them) so the trajectory matches the reference exactly ? do NOT reimplement the decode/tracker.

VM file layout to assume

```
`~/t2/WASB-SBDT` (repo; add `~/t2/WASB-SBDT/src` to sys.path), `~/t2/wasb.pth.tar` (weights),  
`~/t2/g1.mp4` (golden clip mahadevpura_1, 1080p ~59.94fps, ~10448 frames, 16:9),  
`~/t2/g1_traj_baseline.csv` (cached torch-1.11 baseline trajectory `Frame,Visibility,X,Y`).  
Installed: torch 2.6+cu124, torchvision, PyNvVideoCodec 2.1, opencv, numpy, scipy,  
hydra/omegaconf.
```

What exp_t2_full.py must do (unbuffered prints, low memory: stream frames, sliding 3-frame window)

1. Build WASB detector (HRNet + postprocessor via WASB's `build\_detector` OR HRNet + WASB's postprocessor class directly) + the OnlineTracker, load weights, cuda eval.
2. Define a `run(frontend)` that iterates the clip via `frontend` in ('cpu','gpu'), producing per-frame shuttle (visibility,x,y) EXACTLY as the reference does (same affine `trans`, same run_tensor call, same det accumulation, same tracker update order), and writes
`~/t2/{frontend}\_traj.csv` with header `Frame,Visibility,X,Y`:

- **cpu front-end:** cv2.VideoCapture decode -> build the same window tensors the reference builds (RGB, resize/affine to 512x288, ImageNet-normalize) -> run_tensor.
- **gpu front-end:** PyNvVideoCodec `SimpleDecoder(video, gpu\_id=0, output\_color\_type=OutputColorType.RGB)` -> `torch.from\_dlpack` -> GPU resize to 512x288 (F.interpolate bilinear) + GPU ImageNet-normalize -> build windows on GPU -> run_tensor. (If SimpleDecoder RGB path errors, fall back to default + convert; wrap defensively and print the error.)

Print frames processed + fps for each front-end.

3. After both: print a parity summary comparing the two produced trajectories AND each vs the baseline CSV: for frames where BOTH have visibility=1, report median & 95th-pct euclidean pixel distance; also report visibility-agreement rate. Keep it memory-light (accumulate, don't stack).
4. Process the FULL clip (all frames) so the CSVs are directly scoreable against the labels. Wrap each front-end in try/except so one failing doesn't lose the other; flush stdout.

Robustness notes: the tracker is stateful and order-sensitive ? reset it (`refresh()`) before each front-end run. The affine `trans` maps 512x288 heatmap coords back to original 1920x1080 pixels; replicate the reference's exact transform so X,Y are in original resolution (matching the baseline CSV). Confirm which output channel / how the reference picks the per-frame detection (it accumulates candidates per frame id across the 3-frame windows, then the tracker resolves) ? mirror it.

Return the complete `exp\_t2\_full.py` + a short notes section listing: which WASB classes you imported (detector/postprocessor/tracker paths), how you built `trans`, and anything likely to need a tweak on first run.

2. assistant (2026-07-06T20:41:02.546Z)

I'll start by reading the reference implementation and WASB's own code to replicate the exact inference glue.

3. user (2026-07-06T20:41:04.280Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
```

```

runner
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     ""
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63
64     @dataclass
65     class NativeWasbConfig:
66         """Resolved settings for a Linux-native WASB run.
67
68         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71         ""
72
73         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74         python_bin: str = (
75             "python" # the `wasb` conda env python (invoked by path, no activate)
76         )
77         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78         sport: str = "badminton"
79         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir

```

```

81     timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82     keep_frames: bool = (
83         False # retain the decoded frame cache after success (debug only)
84     )
85     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86     # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87     # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88     stream_video: bool = True
89     # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90     # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91     batch_size: int = 0
92     prefetch_batches: int = 0
93
94     @classmethod
95     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96         if isinstance(idx_cfg, dict):
97             idx_cfg = IndexingConfig(**idx_cfg)
98         w = idx_cfg.wasb
99         env = os.environ.get
100         # stream_video is tri-state in config: None/absent => native default (stream);
an
101         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
102         sv = w.stream_video
103         return cls(
104             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
105             python_bin=env("WASB_PYTHON") or w.python_bin,
106             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
107             sport=w.sport,
108             device=env("WASB_DEVICE") or w.device,
109             scratch_dir=env("WASB_SCRATCH")
110             or env("RALLY_SCRATCH")
111             or env("TMPDIR")
112             or "",
113             timeout_sec=w.timeout_sec,
114             keep_frames=w.keep_frames,
115             stream_video=(True if sv is None else bool(sv)),
116             batch_size=int(w.batch_size or 0),
117             prefetch_batches=int(w.prefetch_batches or 0),
118         )
119
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
123
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
127             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
128             self._cuda_available: Optional[bool] = None
129
130         # --- low-level native exec (the single place tests patch)
-----
131         def _run(
132             self, argv: List[str], cwd: Optional[str] = None
133         ) -> subprocess.CompletedProcess:
134             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
135             return subprocess.run(
136                 argv,

```

```

137         cwd=cwd,
138         capture_output=True,
139         text=True,
140         timeout=(self.cfg.timeout_sec or None),
141     )
142
143     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
144     def _probe_cuda(self) -> bool:
145         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
146         (missing python / timeout / import error) is treated as 'no CUDA'."""
147         try:
148             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
149         except (FileNotFoundError, subprocess.TimeoutExpired):
150             return False
151         return res.returncode == 0 and "cuda True" in (res.stdout or "")
152
153     def _cuda_is_available(self) -> bool:
154         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
155         if self._cuda_available is None:
156             self._cuda_available = self._probe_cuda()
157         return self._cuda_available
158
159     def _effective_device(self) -> str:
160         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162         if self.cfg.device == AUTO:
163             return DeviceContext(AUTO, self._cuda_is_available()).effective
164         return self.cfg.device
165
166     def _repo_src(self) -> str:
167         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169     def _copy_infer_script(self) -> bool:
170         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171         repo_src = self._repo_src()
172         try:
173             os.makedirs(repo_src, exist_ok=True)
174             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175             return True
176         except OSError as e:
177             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178         return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182         if path:
183             shutil.rmtree(path, ignore_errors=True)
184
185     def healthcheck(self) -> Tuple[bool, str]:
186         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187         if not self.cfg.weights_path:
188             return False, (
189                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
190             )
191         weights = os.path.expanduser(self.cfg.weights_path)
192         if not os.path.exists(weights):

```

```

193         return False, f"WASB weights not found at {weights}"
194     repo_src = self._repo_src()
195     if not os.path.isdir(repo_src):
196         return False, (
197             f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
199     )
200     # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201     # argparse rc=2 from wasb_infer.py at run time.
202     if self.cfg.device not in VALID_DEVICES:
203         return False, (
204             f"unknown device {self.cfg.device!r} ? valid: {'',
205             '.join(VALID_DEVICES))."
206         )
207     try:
208         res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
209     except FileNotFoundError:
210         return (
211             False,
212             f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
213         )
214     except subprocess.TimeoutExpired:
215         return False, "native torch healthcheck timed out."
216     out = (res.stdout or "").strip()
217     if res.returncode != 0:
218         return (
219             False,
220             f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
221             out).strip()}",
222         )
223     # M4: resolve the device. Cache the probe so run_predict reuses it (no second
224     subprocess).
225     self._cuda_available = "cuda True" in out
226     ctx = DeviceContext(self.cfg.device, self._cuda_available)
227     if ctx.cuda_required_but_missing:
228         # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
229         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
230         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
231     if ctx.fell_back:
232         logger.warning(
233             "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
234             "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
235             out,
236         )
237     return True, out
238
239 def run_predict(
240     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
241 ) -> Optional[str]:
242     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
243
244     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
245     runner does, and the CSV is named ``{stem}_{vid12}_wasb.csv`` ? byte-for-byte
the
246     same downstream contract, so the parity comparison is apples-to-apples.
247     """
248     output_dir = os.path.abspath(output_win_dir)
249     os.makedirs(output_dir, exist_ok=True)
250     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
251     norm = str(video_win_path).replace("\\", "/")

```

```

249         stem, _ext = os.path.splitext(os.path.basename(norm))
250         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
251         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
252         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
253
254         if not self._copy_infer_script():
255             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
256             return None
257
258         weights = os.path.expanduser(self.cfg.weights_path)
259         argv = [
260             self.cfg.python_bin,
261             "wasb_infer.py",
262             "--video",
263             os.path.abspath(str(video_win_path)),
264         ]
265         frames_dir: Optional[str] = None
266         if self.cfg.stream_video:
267             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
268             argv.append("--stream-video")
269         else:
270             scratch = self.cfg.scratch_dir or output_dir
271             frames_dir = os.path.join(scratch, f"{key}_frames")
272             argv += ["--frames_out_dir", frames_dir]
273             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275             device = self._effective_device()
276             argv += [
277                 "--weights",
278                 weights,
279                 "--sport",
280                 self.cfg.sport,
281                 "--device",
282                 device,
283                 "--out",
284                 out_csv,
285             ]
286             # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287             # byte-parity guarantee) is unchanged when the knobs are unset.
288             if self.cfg.batch_size:
289                 argv += ["--batch-size", str(self.cfg.batch_size)]
290             if self.cfg.prefetch_batches:
291                 argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293             logger.info(
294                 "Running native WASB inference (device=%s%s) on %s ...",
295                 device,
296                 " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297                 os.path.basename(norm),
298             )
299             try:
300                 res = self._run(argv, cwd=self._repo_src())
301             except FileNotFoundError:
302                 logger.error(
303                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304                 )
305                 return None
306             except subprocess.TimeoutExpired:
307                 logger.error(
308                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec

```

```

309         )
310         return None
311     if res.returncode != 0:
312         logger.error(
313             "WASB native inference failed:\n%s",
314             (res.stderr or res.stdout or "")[-2000:],
315         )
316         return None
317     if not os.path.exists(out_csv):
318         logger.error(
319             "WASB native finished but expected CSV not found at %s", out_csv
320         )
321         return None
322     logger.info("WASB trajectory CSV: %s", out_csv)
323
324     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327     if not self.cfg.keep_frames and frames_dir:
328         logger.info(
329             "Cache hygiene: removing frame cache for %s "
330             "(set indexing.wasb.keep_frames=true to retain).",
331             key,
332         )
333         self._rm_rf(frames_dir)
334     return out_csv
335
336     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338     trajectory_to_action_windows = staticmethod(
339         TrackNetRunner.trajectory_to_action_windows
340     )
341

```

4. user (2026-07-06T20:41:04.472Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a
18    trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19    resume mid-stream). We exploit that split:
20
21    * The detector pass writes its per-window raw outputs incrementally to
22      ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23      records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24      most ``--flush-every`` windows of GPU work instead of the whole pass.
25    * On restart we replay the cached windows, skip the DataLoader ahead to the first

```

```

26         un-cached window, and continue. Once every window is cached, the detector pass
27         is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28         the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29         not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40
41     """
42     import argparse
43     import csv
44     import glob
45     import json
46     import logging
47     import os
48     import os.path as osp
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
57
58     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
59         """Compose the WASB eval config via Hydra, overriding model/weights/device.
60
61         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
62         preserved),
63         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
64         ``runner.gpus=[0]``;
65         cpu/mps run with no GPU list so the detector places tensors on ``device``.
66         """
67         from hydra import compose, initialize
68
69         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
70         with initialize(config_path="configs", version_base=None):
71             cfg = compose(
72                 config_name="eval",
73                 overrides=[
74                     f"dataset={sport}",
75                     "model=wasb",
76                     f"detector.model_path={weights}",
77                     f"runner.device={device}",
78                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
79         this box
80
81             ],
82             )
83         return cfg
84
85     def _frame_id(path: str) -> int:
86         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
87         180)."""
88         stem = osp.splitext(osp.basename(path))[0]
89         digits = "".join(ch for ch in stem if ch.isdigit())

```

```

85         return int(digits) if digits else -1
86
87
88     # ----- #
89     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
90     # ----- #
91     def _cache_paths(cache_dir: str):
92         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
93
94
95     def default_cache_dir(out_csv: str) -> str:
96         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
97         d = osp.dirname(osp.abspath(out_csv))
98         stem = osp.splitext(osp.basename(out_csv))[0]
99         return osp.join(d, stem + "_wasbcache")
100
101
102     def _atomic_write_text(path: str, text: str) -> None:
103         """Write then rename so a crash mid-write can't leave a half-written file."""
104         tmp = path + ".tmp"
105         with open(tmp, "w") as f:
106             f.write(text)
107             f.flush()
108             os.fsync(f.fileno())
109         os.replace(tmp, path)
110
111
112     def _det_plain(det) -> list:
113         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
114         list."""
115         xy = det["xy"]
116         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
117
118     def _dets_to_tracker(plain_list):
119         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
120         array,
121         the tracker does vector math / np.linalg.norm on it)."""
122         import numpy as np
123
124         return [
125             {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
126             for p in plain_list
127         ]
128
129     def write_manifest(cache_dir: str, manifest: dict) -> None:
130         _, mpath = _cache_paths(cache_dir)
131         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
132
133
134     def read_manifest(cache_dir: str):
135         _, mpath = _cache_paths(cache_dir)
136         if not osp.exists(mpath):
137             return None
138         try:
139             with open(mpath) as f:
140                 return json.load(f)
141         except (json.JSONDecodeError, OSError):
142             return None
143
144
145     def manifest_compatible(
146         manifest: dict,
147         *,

```

```

148     frames_dir: str,
149     weights: str,
150     sport: str,
151     frames_in: int,
152     frames_total: int,
153     device: str = "cuda",
154 ) -> bool:
155     """A cache is reusable only if the run that produced it matches this run's inputs.
156
157     ``device`` defaults to ``cuda`` so caches written before device-keying (no
158     ``device`` field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
159     cache (detections can differ numerically across devices), and vice-versa.
160
161     ``frames_dir`` is the already-canonical cache key the caller stored in the
162     manifest (``run()`` passes its ``cache_key`` here, and writes the same value into the
163     manifest): ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
164     ``video:<abspath>`` key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
165     it here: ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
166     prepended, so a streaming cache would never match itself and every re-run would recompute
167     (issue #348).
168     """
169     if not manifest or manifest.get("version") != CACHE_VERSION:
170         return False
171     return (
172         manifest.get("frames_dir") == frames_dir
173         and manifest.get("weights") == weights
174         and manifest.get("sport") == sport
175         and manifest.get("frames_in") == frames_in
176         and manifest.get("frames_total") == frames_total
177         and manifest.get("device", "cuda") == device
178     )
179
180 def reset_cache(cache_dir: str) -> None:
181     """Drop any existing cache so the next run starts clean."""
182     det_jsonl, mpath = _cache_paths(cache_dir)
183     for p in (det_jsonl, mpath):
184         if osp.exists(p):
185             os.remove(p)
186
187
188 def load_and_clean_cache(cache_dir: str):
189     """Read the longest contiguous (w == line index) valid prefix of det_raw.jsonl,
190     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
191     prefix so a trailing half-written line from a crash can't corrupt future appends.
192
193     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194     [x, y, score, scale] candidates (accumulated across every window the frame is in,
195     in window order ? identical to a non-resumed run).
196     """
197     det_jsonl, _ = _cache_paths(cache_dir)
198     det_by_fid = defaultdict(list)
199     valid: list = []
200     if osp.exists(det_jsonl):
201         with open(det_jsonl, "r") as f:
202             for line in f:
203                 s = line.strip()
204                 if not s:

```

```

205         continue
206     try:
207         obj = json.loads(s)
208     except json.JSONDecodeError:
209         break # truncated trailing line (crash mid-flush)
210     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
211         break
212     valid.append(s)
213     for fid, plain in obj["f"]:
214         det_by_fid[int(fid)].extend([list(p) for p in plain])
215     # Guarantee a clean append boundary by rewriting only the good prefix.
216     _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
217 return len(valid), det_by_fid
218
219
220 def _append_windows(fh, objs) -> None:
221     """Append window records as JSONL and durably flush (bounded loss on crash)."""
222     for o in objs:
223         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
224     fh.flush()
225     os.fsync(fh.fileno())
226
227
228 def _atomic_write_trajectory(out_csv: str, rows) -> None:
229     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
230     tmp = out_csv + ".tmp"
231     with open(tmp, "w", newline="") as f:
232         w = csv.writer(f)
233         w.writerow(["Frame", "Visibility", "X", "Y"])
234         w.writerows(rows)
235         f.flush()
236         os.fsync(f.fileno())
237     os.replace(tmp, out_csv)
238
239
240 # ----- #
241 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242 # ----- #
243 _STREAM_FRAME_FMT = "{:08d}.png"
244
245
246 def synthetic_frame_path(index: int) -> str:
247     """Stable, index-encoding frame name used by the streaming path.
248
249     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
250     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
251     frames came from disk or from the in-memory stream. The streaming image loader
252     inverts this name back to an index to fetch the decoded frame.
253     """
254     return _STREAM_FRAME_FMT.format(int(index))
255
256
257 def build_windows(frames: list, frames_in: int):
258     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
259     + checkpoint). Pure list math, identical for the disk and streaming paths.
260
261     Returns a list of windows, each a list of ``frames_in`` consecutive paths
262     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
263     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
264     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
265     """
266     return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
267
268
269 class SequentialFrameStore:

```

```

270     """In-memory sliding buffer over a forward-only frame reader, sized for the
271     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279     earliest index any not-yet-finished window can still ask for) and decode each source
280     frame exactly once via the forward-only ``reader(index)``.
281     """
282
283     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284         self._reader = reader
285         self._total = int(total)
286         self._window = max(1, int(window))
287         self._buf: dict = {}
288         # On a resumed run the detector's first needed frame is `start_index`; begin
289         # pulling there so the reader can skip (seek past) the already-cached prefix
290         # instead of decoding 0..start_index-1 just to throw them away.
291         self._next = int(start_index) # next index not yet pulled from the reader
292         self._max_seen = int(start_index) - 1
293         self._floor = int(start_index) # lowest index we still keep (never evict below)
294
295     def get(self, index: int):
296         index = int(index)
297         if index < self._floor:
298             raise KeyError(
299                 f"frame {index} already evicted (below floor {self._floor}); the access
300                 "
301                 f"pattern must stay within a window of the highest frame seen"
302             )
303         if index >= self._total:
304             raise KeyError(f"frame {index} out of range (total {self._total})")
305         # Pull forward from the reader until the requested index is buffered.
306         while self._next <= index:
307             self._buf[self._next] = self._reader(self._next)
308             self._next += 1
309         if index > self._max_seen:
310             self._max_seen = index
311         # Evict frames older than the earliest a still-running window could re-request:
312         # once we've seen index `m`, no future window starts before `m - (window-1)`.
313         new_floor = max(self._floor, self._max_seen - (self._window - 1))
314         for k in range(self._floor, new_floor):
315             self._buf.pop(k, None)
316         self._floor = new_floor
317         return self._buf[index]
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326
327     def _bgr_to_pil_rgb(frame_bgr):
328         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
329         EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->

```

```

Image.open(PNG).convert('RGB'))`;
333     because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
334     bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
335     frame indistinguishable from the on-disk one to everything downstream.
336     """
337     import cv2
338     from PIL import Image
339
340     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
341
342
343     def _make_streamed_read_image(store, real_read_image):
344         """Build the ``read_image`` replacement used during a streaming detector pass.
345
346         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
347         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
348         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
349         without the WSL-only ``dataloaders`` package.
350         """
351
352         def _read_image(path, *args, **kwargs):
353             idx = _index_from_synthetic_path(path)
354             if idx < 0:
355                 return real_read_image(path, *args, **kwargs)
356             return _bgr_to_pil_rgb(store.get(idx))
357
358         return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
366         frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
370         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
371         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
372         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
373         ``osp.exists`` guard is sidestepped for synthetic stream paths.
374
375         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
376         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
377         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
378         reference).
379
380         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
381         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
382         resume we start at ``start_index`` so the decoder seeks past already-cached windows.

```

```

The
383     original reader is restored on exit.
384     """
385     from dataloaders import dataset_loader as _dl
386
387     store = SequentialFrameStore(
388         frame_loader, total, window=window, start_index=start_index
389     )
390     real_read_image = _dl.read_image
391     # Rebind read_image in the consuming module for the duration of the detector pass;
392     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
393     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
394     try:
395         yield store
396     finally:
397         _dl.read_image = real_read_image # type: ignore[assignment]
398
399
400     # ----- #
401     # Inference
402     # ----- #
403     class _PrefetchIter:
404         """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).
405
406         In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
407         the cv2.imread shim live in this process and cannot be pickled to workers), so the
408         CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
409         measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410
411         A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
412         release the GIL for the heavy parts, the store's forward-only access pattern is
413         preserved (one consumer thread iterating sequentially), and the bounded queue caps
414         memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
415
416         _END = object()
417
418         def __init__(self, src, depth: int = 2):
419             import queue
420             import threading
421
422             self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
423             self._exc: "BaseException | None" = None
424
425             def _fill() -> None:
426                 try:
427                     for item in src:
428                         self._q.put(item)
429                     except BaseException as e: # noqa: BLE001 - surfaced in __next__
430                         self._exc = e
431                     finally:
432                         self._q.put(self._END)
433
434             self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
435             self._t.start()
436
437         def __iter__(self):
438             return self
439
440         def __next__(self):
441             item = self._q.get()
442             if item is self._END:

```

```

442         if self._exc is not None:
443             raise self._exc
444         raise StopIteration
445     return item
446
447
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
453         limit: int = 0,
454         batch_size: int = 8,
455         num_workers: int = 4,
456         log_every_batches: int = 50,
457         cache_dir: "str | None" = None,
458         flush_every: int = 1000,
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",
463         prefetch_batches: int = 0,
464     ) -> str:
465         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
471           ``cv2.imread``.
472         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
473           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
474           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
475           reader, which returns a PIL RGB image) over the DataLoader pass so every other
476           line of
477           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
478           identical
479           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
480           ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
481           ``num_workers=0``
482           (the in-process shim + buffer can't cross worker processes).
483
484           ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
485           frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
486           after reboot still matches.
487         """
488         import time
489
490         import torch
491         from dataloaders import build_img_transforms, build_seq_transforms
492         from dataloaders.dataset_loader import ImageDataset
493         from torch.utils.data import DataLoader
494
495         # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
496         the
497         # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
498         suppresses
499         # the mypy false positive where the main env resolves `trackers` to the Roboflow
500         package
501         # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
502         separate
503         # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
504         dep.
505         from trackers import build_tracker # type: ignore[attr-defined]
506         from utils import Center

```

```

498
499     from detectors import build_detector
500
501     streaming = frame_loader is not None
502
503     cfg = _build_cfg(weights, sport, device)
504     frames_in = int(cfg["model"]["frames_in"])
505     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508     if streaming:
509         # `frames` is already the ordered list of (synthetic) frame paths.
510         if not isinstance(frames, (list, tuple)):
511             raise SystemExit(
512                 "streaming mode requires `frames` to be a list of frame paths"
513             )
514         frames = list(frames)
515         frames_dir = source_key or "stream"
516     else:
517         frames_dir = frames
518         frames = sorted(
519             glob.glob(osp.join(frames_dir, "*.png"))
520             + glob.glob(osp.join(frames_dir, "*.jpg"))
521         )
522     if limit:
523         frames = frames[:limit]
524     if len(frames) < frames_in:
525         where = frames_dir if not streaming else "video stream"
526         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
529     checkpoint).
530     samples = []
531     for window in build_windows(frames, frames_in):
532         annos = [
533             {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
534             for p in window
535         ]
536         samples.append({"frames": window, "annos": annos})
537     windows_total = len(samples)
538
539     # ---- cache setup / resume decision ----- #
540     if cache_dir is None:
541         cache_dir = default_cache_dir(out_csv)
542         os.makedirs(cache_dir, exist_ok=True)
543         det_jsonl, _ = _cache_paths(cache_dir)
544         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
545         manifest = None if fresh else read_manifest(cache_dir)
546         resume = manifest_compatible(
547             manifest or {},
548             frames_dir=cache_key,
549             weights=weights,
550             sport=sport,
551             frames_in=frames_in,
552             frames_total=len(frames),
553             device=device,
554         )
555     if resume:
556         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
557         logger.info(
558             f"resuming from cache: {windows_done}/{windows_total} windows already
559             detected"
560         )
561     else:
562         if manifest is not None:

```

```

561         logger.info("cache incompatible with this run -> starting fresh")
562         reset_cache(cache_dir)
563         windows_done, det_by_fid = 0, defaultdict(list)
564
565     base_manifest = {
566         "version": CACHE_VERSION,
567         "frames_dir": cache_key,
568         "weights": weights,
569         "sport": sport,
570         "frames_in": frames_in,
571         "frames_total": len(frames),
572         "device": device,
573         "windows_total": windows_total,
574         "windows_done": windows_done,
575     }
576     write_manifest(cache_dir, base_manifest)
577
578     # ---- detector pass over the un-cached windows ----- #
579     remaining = samples[windows_done:]
580     if remaining:
581         _, transform_test = build_img_transforms(cfg)
582         try:
583             _, seq_transform_test = build_seq_transforms(cfg)
584         except Exception:
585             seq_transform_test = None
586         ds = ImageDataset(
587             cfg,
588             remaining,
589             input_wh,
590             output_wh,
591             transform=transform_test,
592             seq_transform=seq_transform_test,
593             is_train=False,
594         )
595         # Streaming forces single-process loading: the in-memory frame store + the
596         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
597         loader_workers = 0 if streaming else num_workers
598         loader = DataLoader(
599             ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600         )
601         detector = build_detector(cfg)
602
603         from contextlib import ExitStack
604
605         t0 = time.time()
606         done = windows_done
607         win_idx = windows_done
608         log_every = max(1, log_every_batches)
609         flush_n = max(1, flush_every)
610         pending = []
611         logger.info(
612             f"detecting on {len(remaining)} remaining windows "
613             f"total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614             f"source={'video-stream' if streaming else 'frames-dir'}..."
615         )
616         det_f = open(det_jsonl, "a")
617         try:
618             with ExitStack() as stack:
619                 stack.enter_context(torch.no_grad())
620                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
621                 # in-memory decoded frame for that synthetic path; every other line of
622                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
623                 exactly.

```

```

623         # The detector's first needed frame is `windows_done` (window i starts
at
624         # frame i), so prime the store there for a resume.
625         if streaming:
626             stack.enter_context(
627                 _streaming_read_image_patch(
628                     frame_loader,
629                     len(frames),
630                     window=frames_in,
631                     start_index=windows_done,
632                 )
633             )
634         # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635         # a multi-worker DataLoader already overlaps via its worker processes).
636         batches = (
637             _PrefetchIter(loader, depth=prefetch_batches)
638             if prefetch_batches and loader_workers == 0
639             else loader
640         )
641         for bi, batch in enumerate(batches):
642             imgs, _hms, trans = batch[0], batch[1], batch[2]
643             img_paths = [
644                 list(t) for t in batch[-1]
645             ] # [frame_in][batch] -> path
646             batch_results, _ = detector.run_tensor(imgs, trans)
647             nb = imgs.shape[0]
648             for ib in range(nb):
649                 frames_payload = []
650                 for ie in sorted(batch_results[ib].keys()):
651                     p = img_paths[ie][ib]
652                     fid = _frame_id(p)
653                     plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                     frames_payload.append([fid, plain])
655                     det_by_fid[fid].extend(plain)
656                 pending.append({"w": win_idx, "f": frames_payload})
657                 win_idx += 1
658             done += nb
659             if len(pending) >= flush_n or done >= windows_total:
660                 _append_windows(det_f, pending)
661                 pending = []
662                 base_manifest["windows_done"] = done
663                 write_manifest(cache_dir, base_manifest)
664             if (bi + 1) % log_every == 0 or done >= windows_total:
665                 el = time.time() - t0
666                 new_done = done - windows_done
667                 rate = new_done / el if el > 0 else 0.0
668                 eta = (windows_total - done) / rate if rate > 0 else 0.0
669                 logger.info(
670                     f"{done}/{windows_total} "
671                     f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
672                     f"elapsed {el:.0f}s | ETA {eta:.0f}s"
673                 )
674             if pending:
675                 _append_windows(det_f, pending)
676                 base_manifest["windows_done"] = done
677                 write_manifest(cache_dir, base_manifest)
678         finally:
679             det_f.close()
680             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
681     else:
682         logger.info("detector cache already complete -> tracker-only re-run")
683
684     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #

```

```

685     tracker = build_tracker(cfg)
686     tracker.refresh()
687     rows = []
688     for fid in sorted(det_by_fid.keys()):
689         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
690         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
691
692     _atomic_write_trajectory(out_csv, rows)
693     visible = sum(1 for _, v, *_ in rows if v)
694     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
695     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
696     return out_csv
697
698
699 def extract_frames(video_path: str, frames_dir: str) -> str:
700     """Decode a video to PNG frames named by source frame index (00000000.png ...).
701
702     Idempotent: if the dir already holds the expected number of frames (matching the
703     container's reported frame count) we skip re-decoding ? re-extraction after a
704     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
705     extracting with ffmpeg on the host and passing --frames_dir.
706     """
707     import cv2
708
709     os.makedirs(frames_dir, exist_ok=True)
710     cap = cv2.VideoCapture(video_path)
711     if not cap.isOpened():
712         raise SystemExit(f"cannot open video: {video_path}")
713     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
714     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
715     if expected > 0 and existing >= expected:
716         cap.release()
717         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
718         return frames_dir
719     i = 0
720     while True:
721         ok, frame = cap.read()
722         if not ok:
723             break
724         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
725         i += 1
726     cap.release()
727     logger.info(f"extracted {i} frames -> {frames_dir}")
728     return frames_dir
729
730
731 def _probe_frame_count(video_path: str) -> int:
732     """Container-reported frame count via cv2 (0 if unknown). Used to size the
733     stream."""
734     import cv2
735
736     cap = cv2.VideoCapture(video_path)
737     if not cap.isOpened():
738         raise SystemExit(f"cannot open video: {video_path}")
739     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
740     cap.release()
741     return n
742
743 def make_video_frame_loader(video_path: str):
744     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
745
746     ``frame_loader(index)`` -> BGR uint8 ndarray`` decodes the video forward-only with one
747     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748     detector's sliding-window order); it reads sequentially and only uses a frame-seek

```

```

to
749     skip forward when a *resume* starts past the current position. Each ``cap.read()``
750     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
751     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
752
753     Returns the count hint from the container so the caller can build the frame list;
754     ``release`` closes the capture.
755     """
756     import cv2
757
758     cap = cv2.VideoCapture(video_path)
759     if not cap.isOpened():
760         raise SystemExit(f"cannot open video: {video_path}")
761     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
762     state = {"pos": 0} # next index cap.read() will return
763
764     def frame_loader(index: int):
765         index = int(index)
766         if index < state["pos"]:
767             raise RuntimeError(
768                 f"streaming decode is forward-only; got index {index} after
{state['pos']}"
769             )
770         if index > state["pos"]:
771             # Forward skip only happens when resuming past cached windows. Seek once.
772             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
773             state["pos"] = index
774         ok, frame = cap.read()
775         if not ok:
776             raise RuntimeError(f"failed to decode frame {index} from {video_path}")
777         state["pos"] += 1
778         return frame
779
780     def release():
781         cap.release()
782
783     return frame_loader, count_hint, release
784
785
786 def run_video_streaming(
787     video_path: str,
788     weights: str,
789     out_csv: str,
790     sport: str = "badminton",
791     limit: int = 0,
792     batch_size: int = 8,
793     log_every_batches: int = 50,
794     cache_dir: "str | None" = None,
795     flush_every: int = 1000,
796     fresh: bool = False,
797     device: str = "cuda",
798     prefetch_batches: int = 0,
799 ) -> str:
800     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
801     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
802     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
803     ``run``'s docstring for why.
804     """
805     frame_loader, count_hint, release = make_video_frame_loader(video_path)
806     try:
807         if count_hint <= 0:
808             raise SystemExit(
809                 f"could not determine frame count for {video_path}; cannot stream "
810                 f"(fall back to frame extraction with --frames_dir)"

```

```

811         )
812         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
813         source_key = "video:" + osp.abspath(video_path)
814         return run(
815             frame_list,
816             weights,
817             out_csv,
818             sport,
819             limit,
820             batch_size=batch_size,
821             num_workers=0,
822             log_every_batches=log_every_batches,
823             cache_dir=cache_dir,
824             flush_every=flush_every,
825             fresh=fresh,
826             frame_loader=frame_loader,
827             source_key=source_key,
828             device=device,
829             prefetch_batches=prefetch_batches,
830         )
831     finally:
832         release()
833
834
835 def main():
836     ap = argparse.ArgumentParser()
837     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
838     ap.add_argument(
839         "--video", help="video file; frames are extracted internally (in WSL)"
840     )
841     ap.add_argument(
842         "--frames_out_dir", help="where to extract frames when --video is used"
843     )
844     ap.add_argument(
845         "--stream-video",
846         action="store_true",
847         help="FAST PATH: decode --video on the fly and feed frames straight to "
848         "the detector (no PNG extraction). Output-equivalent to the disk path.",
849     )
850     ap.add_argument("--weights", required=True)
851     ap.add_argument("--out", required=True)
852     ap.add_argument("--sport", default="badminton")
853     ap.add_argument(
854         "--device",
855         default="cuda",
856         choices=["cuda", "mps", "cpu"],
857         help="torch device for the detector (default cuda = the WSL parity path)",
858     )
859     ap.add_argument(
860         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
861     )
862     ap.add_argument("--batch-size", type=int, default=8)
863     ap.add_argument("--num-workers", type=int, default=4)
864     ap.add_argument(
865         "--prefetch-batches",
866         type=int,
867         default=0,
868         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
869         "bounded producer thread of this queue depth (0 = off, the parity default)",
870     )
871     ap.add_argument(
872         "--log-every-batches", type=int, default=50, help="progress log cadence"
873     )
874     ap.add_argument(
875         "--cache-dir",

```

```

876         default=None,
877         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
878     )
879     ap.add_argument(
880         "--flush-every",
881         type=int,
882         default=1000,
883         help="fsync the detector cache every N windows (bounds reboot loss)",
884     )
885     ap.add_argument(
886         "--fresh", action="store_true", help="ignore any existing cache and start over"
887     )
888     args = ap.parse_args()
889
890     if args.stream_video:
891         if not args.video:
892             raise SystemExit("--stream-video requires --video")
893         run_video_streaming(
894             args.video,
895             args.weights,
896             args.out,
897             args.sport,
898             args.limit,
899             batch_size=args.batch_size,
900             log_every_batches=args.log_every_batches,
901             cache_dir=args.cache_dir,
902             flush_every=args.flush_every,
903             fresh=args.fresh,
904             device=args.device,
905             prefetch_batches=args.prefetch_batches,
906         )
907         return
908
909     frames_dir = args.frames_dir
910     if args.video:
911         frames_dir = extract_frames(
912             args.video, args.frames_out_dir or (args.video + "_frames")
913         )
914     if not frames_dir:
915         raise SystemExit("provide --frames_dir or --video")
916     run(
917         frames_dir,
918         args.weights,
919         args.out,
920         args.sport,
921         args.limit,
922         batch_size=args.batch_size,
923         num_workers=args.num_workers,
924         log_every_batches=args.log_every_batches,
925         cache_dir=args.cache_dir,
926         flush_every=args.flush_every,
927         fresh=args.fresh,
928         device=args.device,
929     )
930
931
932 if __name__ == "__main__":
933     main()
934

```

5. assistant (2026-07-06T20:41:08.091Z)

Let me read the WASB source code to replicate the exact inference glue.

6. user (2026-07-06T20:41:10.745Z)

```

1     import os.path as osp
2     import logging
3     from collections import defaultdict
4     from hydra.core.hydra_config import HydraConfig
5     import numpy as np
6     from PIL import Image
7     import cv2
8     import torch
9     from torch import nn
10
11     from models import build_model
12     from dataloaders import read_image, get_transform, build_img_transforms
13     from utils.image import get_affine_transform, affine_transform
14     from .postprocessor import TracknetV2Postprocessor
15     from .deepball_postprocessor import DeepBallPostprocessor
16
17     log = logging.getLogger(__name__)
18
19     class TracknetV2Detector(object):
20
21         __postprocessor_factory = {
22             'tracknetv2': TracknetV2Postprocessor,
23             'deepball': DeepBallPostprocessor,
24         }
25
26         def __init__(self, cfg, model=None):
27             self._frames_in = cfg['model']['frames_in']
28             self._frames_out = cfg['model']['frames_out']
29             self._input_wh = (cfg['model']['inp_width'], cfg['model']['inp_height'])
30
31             self._scales = cfg['detector']['postprocessor']['scales']
32
33             self._2d_input = True
34             model_name = cfg['model']['name']
35             if model_name in ['tracknetv2', 'hrnet', 'monotrack', 'retracknetv2',
36 'deepball', 'ballseg']:
37                 pass
38             else:
39                 raise ValueError('unknown model_name : {}'.format(model_name))
40
41             _, self._transform = build_img_transforms(cfg)
42
43             self._device = cfg['runner']['device']
44             if self._device != 'cuda':
45                 assert 0, 'device=cpu not supported'
46             if not torch.cuda.is_available():
47                 assert 0, 'GPU NOT available'
48             self._gpus = cfg['runner']['gpus']
49
50             if model is None:
51                 self._model = build_model(cfg)
52                 model_path = cfg['detector']['model_path']
53                 if model_path is None:
54                     output_dir = HydraConfig.get().run_dir
55                     model_path = osp.join(output_dir, 'best_model.pth.tar')
56                     log.info('Checkpoint is not specified, so it is set as the best model in
57 {}'.format(output_dir))
58                     if not osp.exists(model_path):
59                         FileNotFoundError('{} not found'.format(model_path))
60                     checkpoint = torch.load(model_path)
61                     self._model.load_state_dict(checkpoint['model_state_dict'])
62                     self._model = self._model.to(self._device)
63                     self._model = nn.DataParallel(self._model, device_ids=self._gpus)
64             else:
65                 self._model = model

```

```

64         self._model.eval()
65
66         postprocessor_name = cfg['detector']['postprocessor']['name']
67         if not postprocessor_name in self.__postprocessor_factory.keys():
68             raise KeyError('invalid dataset: {}'.format(postprocessor_name ))
69         self._postprocessor = self.__postprocessor_factory[postprocessor_name](cfg)
70
71     @property
72     def frames_in(self):
73         return self._frames_in
74
75     @property
76     def frames_out(self):
77         return self._frames_out
78
79     @property
80     def input_wh(self):
81         return self._input_wh
82
83     def run_tensor(self, imgs, affine_mats):
84         imgs = imgs.to(self._device)
85         preds = self._model(imgs)
86         pp_results = self._postprocessor.run(preds, affine_mats)
87
88         results = {}
89         hms_vis = {}
90         for bid in sorted(pp_results.keys()):
91             results[bid] = {}
92             hms_vis[bid] = {}
93             for eid in sorted(pp_results[bid].keys()):
94                 results[bid][eid] = []
95                 hms_vis[bid][eid] = []
96                 for scale in sorted(pp_results[bid][eid].keys()):
97                     scores = pp_results[bid][eid][scale]['scores']
98                     xys = pp_results[bid][eid][scale]['xys']
99                     for xy, score in zip(xys, scores):
100                         results[bid][eid].append({ 'xy': xy, 'score': score, 'scale':
101 scale})
102
103                     hm = pp_results[bid][eid][scale]['hm']
104                     trans = pp_results[bid][eid][scale]['trans']
105                     hms_vis[bid][eid].append({'hm': hm, 'scale': scale, 'trans': trans})
106
107         return results, hms_vis
108
109

```

7. user (2026-07-06T20:41:11.292Z)

```

1     from collections import defaultdict
2     import numpy as np
3     import cv2
4     import torch
5
6     from utils.utils import _nms, _top1
7     from utils.image import get_affine_transform, affine_transform
8
9     class TracknetV2Postprocessor(object):
10         def __init__(self, cfg):
11             #print(cfg['detector']['postprocessor'])
12             self._score_threshold = cfg['detector']['postprocessor']['score_threshold']
13             self._model_name = cfg['model']['name']
14             self._scales = cfg['detector']['postprocessor']['scales']
15             self._blob_det_method = cfg['detector']['postprocessor']['blob_det_method']

```

```

16         self._use_hm_weight = cfg['detector']['postprocessor']['use_hm_weight']
17         #self._xy_comp_method = cfg['detector']['postprocessor']['xy_comp_method']
18         #print(self._score_threshold, self._scales)
19
20         #self._hm_type = cfg['target_generator']['type']
21         #self._sigmas = cfg['target_generator']['sigmas']
22         self._sigmas = cfg['data_loader']['heatmap']['sigmas']
23         #self._mags = cfg['target_generator']['mags']
24         #self._min_values = cfg['target_generator']['min_values']
25         #print(hm_type, sigmas, mags, min_values)
26
27         """
28         def _detect_blob_center(self, hm):
29             xy = np.array([0., 0.])
30             visi = False
31             max_blob_score = -1
32             if np.max(hm) > self._score_threshold:
33                 visi = True
34                 th, hm_th = cv2.threshold(hm, self._score_threshold, 1,
cv2.THRESH_BINARY)
35                 n_labels, labels = cv2.connectedComponents(hm_th.astype(np.uint8))
36                 for m in range(1, n_labels):
37                     ys, xs = np.where( labels==m )
38                     score = xs.shape[0]
39                     #print(xs, ys)
40                     if score > max_blob_score:
41                         max_blob_score = score
42                         xy = np.array([np.mean(xs), np.mean(ys)])
43                         #xy = np.array([np.unique(xs).mean(), np.unique(ys).mean()])
44             return xy, visi, max_blob_score
45         """
46
47         def _detect_blob_concomp(self, hm):
48             xys, scores = [], []
49             if np.max(hm) > self._score_threshold:
50                 visi = True
51                 th, hm_th = cv2.threshold(hm, self._score_threshold, 1,
cv2.THRESH_BINARY)
52                 n_labels, labels = cv2.connectedComponents(hm_th.astype(np.uint8))
53                 for m in range(1, n_labels):
54                     ys, xs = np.where( labels==m )
55                     ws = hm[ys, xs]
56                     if self._use_hm_weight:
57                         score = ws.sum()
58                         x = np.sum( np.array(xs) * ws ) / np.sum(ws)
59                         y = np.sum( np.array(ys) * ws ) / np.sum(ws)
60                     else:
61                         score = ws.shape[0]
62                         x = np.sum( np.array(xs) ) / ws.shape[0]
63                         y = np.sum( np.array(ys) ) / ws.shape[0]
64                     #print(xs, ys)
65                     #print(score, x, y)
66                     xys.append( np.array([x, y]) )
67                     scores.append( score )
68             return xys, scores
69
70         def _detect_blob_nms(self, hm, sigma):
71             xys, scores = [], []
72             hm_ori = hm.copy()
73             hm_h, hm_w = hm.shape
74             map_x, map_y = np.meshgrid(np.linspace(1, hm_w, hm_w), np.linspace(1, hm_h,
hm_h))
75             while True:
76                 cy, cx = np.unravel_index(np.argmax(hm), hm.shape)
77                 if hm[cy, cx] <= self._score_threshold:

```

```

78         break
79         #dist_map = ((map_y - cy)**2) + ((map_x - cx)**2)
80         dist_map = ((map_y - (cy+1))**2) + ((map_x - (cx+1))**2)
81         ys, xs = np.where( dist_map<=sigma**2)
82         ws = hm_ori[dist_map <= sigma**2]
83         if self._use_hm_weight:
84             score = ws.sum()
85             x = np.sum( np.array(xs) * ws ) / np.sum(ws)
86             y = np.sum( np.array(ys) * ws ) / np.sum(ws)
87         else:
88             score = ws.shape[0]
89             x = np.sum( np.array(xs) ) / ws.shape[0]
90             y = np.sum( np.array(ys) ) / ws.shape[0]
91             #print(xs, ys)
92             #print(score, x, y)
93             xys.append( np.array([x, y]) )
94             scores.append( score)
95             hm[dist_map<=sigma**2] = 0.
96         return xys, scores
97
98     def run(self, preds, affine_mats):
99         results = defaultdict(lambda: defaultdict(dict))
100         for scale in self._scales:
101             preds_ = preds[scale]
102             affine_mats_ = affine_mats[scale].cpu().numpy()
103             hms_ = preds_.sigmoid().cpu().numpy()
104
105             b,s,h,w = hms_.shape
106             for i in range(b):
107                 for j in range(s):
108                     #print(i,j)
109                     """
110                     if self._xy_comp_method=='center':
111                         assert 0, 'not yet'
112                         #xy_, visi_, blob_size_ = self._detect_blob_center(hms_[i,j])
113                         xy_, visi_, blob_score_ = self._detect_blob_center(hms_[i,j])
114                     elif self._xy_comp_method=='gravity':
115                         #xy_, visi_, blob_size_ = self._detect_blob_gravity(hms_[i,j])
116                         #xy_, visi_, blob_score_ = self._detect_blob_gravity(hms_[i,j])
117                         xys_, scores_ = self._detect_blob_gravity(hms_[i,j])
118                     """
119                     if self._blob_det_method=='concomp':
120                         xys_, scores_ = self._detect_blob_concomp(hms_[i,j])
121                     elif self._blob_det_method=='nms':
122                         xys_, scores_ = self._detect_blob_nms(hms_[i,j],
self._sigmas[scale])
123                     else:
124                         raise ValueError('undefined xy_comp_method:
{}'.format(self._xy_comp_method))
125                     xys_t_ = []
126                     for xy_ in xys_:
127                         xys_t_.append( affine_transform(xy_, affine_mats_[i]))
128                     #results[i][j][scale] = {'xy': xy_, 'visi': visi_, 'blob_size':
blob_size_}
129                     #results[i][j][scale] = {'xy': xy_, 'visi': visi_, 'blob_score':
blob_score_}
130                     results[i][j][scale] = {'xys': xys_t_, 'scores': scores_, 'hm':
hms_[i,j], 'trans': affine_mats_[i]}
131
132             #print(results)
133             return results
134
135

```

Now let me read the tracker, dataset loader (transforms + affine), model builder, and configs.

9. user (2026-07-06T20:41:19.678Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\trackers\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\trackers\intra_frame_peak.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\trackers\online.py
```

10. user (2026-07-06T20:41:20.684Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\dataset_loader.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\heatmaps\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\heatmaps\heatmaps.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\img_transforms.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\samplers\samplers.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\samplers\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\data loaders\seq_transforms.py
```

11. user (2026-07-06T20:41:21.678Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\ballseg.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\deepball.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\hrnet.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\monotrack.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\resnetv1b.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\resnet2d.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\segbase.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\unet2d.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\models\unet2d_parts.py
```

12. user (2026-07-06T20:41:22.624Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
```

[illegible]

```

indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\configs\model\wasb.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\optimizer\adadelta_constant.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\optimizer\adadelta_multistep.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\optimizer\adam_multistep.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\src\configs\runner\eval.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\runner\extract_frame.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\tracker\intra_frame_peak.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\tracker\online.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\transform\color.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\transform\crop.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\transform\default.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\transform\full.yaml
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-
SBDT\src\configs\transform\hflip.yaml

```

13. assistant (2026-07-06T20:41:28.161Z)

Let me read the tracker, dataset loader, model builder, image utils, and key configs.

14. user (2026-07-06T20:41:29.198Z)

```

1      import numpy as np
2
3      class Track:
4          def __init__(self):
5              self._xy_dict = {}
6              self._score_dict = {}
7              self._visi_dict = {}
8
9          def add(self, fid, x, y, visi, score):
10             self._xy_dict[fid] = np.array([x,y])
11             self._visi_dict[fid] = visi
12             self._score_dict[fid] = score
13
14          def is_visible(self, fid):
15             if not fid in self._visi_dict.keys():
16                 #raise KeyError('fid {} not found'.format(fid))
17                 return False
18             return self._visi_dict[fid]
19
20          @property
21          def last_fid(self):
22             fids = list( self._xy_dict.keys() )

```

```

23         return max(fids)
24
25     def xy(self, fid):
26         if not fid in self._xy_dict.keys():
27             raise KeyError('fid {} not found'.format(fid))
28         return self._xy_dict[fid]
29
30     def predict(self, last_fid):
31         fid1 = last_fid
32         fid2 = fid1 - 1
33         fid3 = fid1 - 2
34         if self.is_visible(fid1) and self.is_visible(fid2) and self.is_visible(fid3):
35             xy1 = self._xy_dict[fid1]
36             xy2 = self._xy_dict[fid2]
37             xy3 = self._xy_dict[fid3]
38             acc = (xy1-xy2) - (xy2-xy3)
39             vel = (xy1-xy2) + acc
40             xy_pred = xy1 + vel + acc / 2
41         else:
42             xy_pred = None
43         return xy_pred
44
45     class OnlineTracker:
46         def __init__(self, cfg):
47             self._max_disp = cfg['tracker']['max_disp']
48             self._fid = 0
49             self._track = Track()
50
51         def _select_best(self, frame_dets):
52             best_score = - np.Inf
53             visi = False
54             x, y = - np.Inf, - np.Inf
55
56             xy_pred = None
57
58             for det in frame_dets:
59                 score = det['score']
60                 if xy_pred is not None:
61                     qscore = self._compute_quality(xy_pred, det['xy'],
self._track.xy(self._fid-1) )
62                     score += qscore
63
64                 if score > best_score:
65                     best_score = score
66                     xy = det['xy']
67                     x,y = xy[0], xy[1]
68                     visi = True
69             return x,y,visi,best_score
70
71         def _select_not_too_far(self, frame_dets):
72             if (self._fid==0) or (not self._track.is_visible(self._fid-1)):
73                 return frame_dets
74
75             frame_dets_ = []
76             for det in frame_dets:
77                 if np.linalg.norm( det['xy'] - self._track.xy(self._fid-1) ) <
self._max_disp:
78                 frame_dets_.append(det)
79             return frame_dets_
80
81         def _compute_quality(self, xy1, xy2, xy3):
82             return - np.linalg.norm( xy1-xy2 )
83
84         def update(self, frame_dets):
85             frame_dets = self._select_not_too_far(frame_dets)

```

```

86         x,y,visi,score = self._select_best(frame_dets)
87         self._track.add(self._fid, x, y, visi, score)
88
89         self._fid += 1
90         return {'x': x, 'y': y, 'visi': visi, 'score': score}
91
92     def refresh(self):
93         self._fid = 0
94         self._track = Track()
95
96

```

15. user (2026-07-06T20:41:30.194Z)

```

1     from .intra_frame_peak import IntraFramePeakTracker
2     from .online import OnlineTracker
3
4     __tracker_factory = {
5         'intra_frame_peak': IntraFramePeakTracker,
6         'online': OnlineTracker,
7     }
8
9     def build_tracker(cfg):
10         tracker_name = cfg['tracker']['name']
11         if tracker_name not in __tracker_factory.keys():
12             raise KeyError('unknown tracker: {}'.format(tracker_name))
13         return __tracker_factory[tracker_name](cfg)
14
15

```

16. user (2026-07-06T20:41:31.364Z)

```

1     import os.path as osp
2     import json
3     import logging
4     import random
5     from collections import defaultdict
6     import numpy as np
7     import functools
8     from PIL import Image
9     import cv2
10    import torch
11    from torch.utils.data import Dataset
12    import torchvision.transforms.functional as TF
13
14    from utils import read_image
15    from utils.image import get_affine_transform, affine_transform
16    from .heatmaps import select_heatmap_generator
17
18    log = logging.getLogger(__name__)
19
20    def get_transform(img, input_wh, inv=0):
21        h,w,_ = img.shape
22        c = np.array([w / 2., h / 2.], dtype=np.float32)
23        s = max(h, w) * 1.0
24        input_w, input_h = input_wh
25        trans = get_affine_transform(c, s, 0, [input_w, input_h], inv=inv)
26        return trans
27
28    def get_color_jitter_factors(brightness, contrast, saturation, hue):
29        brightness_factor = np.random.uniform(max(0,1-brightness), 1+brightness)
30        contrast_factor = np.random.uniform(max(0,1-contrast), 1+contrast)
31        saturation_factor = np.random.uniform(max(0,1-saturation), 1+saturation)
32        hue_factor = np.random.uniform(-hue, hue)
33        return brightness_factor, contrast_factor, saturation_factor, hue_factor

```

```

34
35 class ImageDataset(Dataset):
36     def __init__(self,
37                 cfg,
38                 dataset,
39                 input_wh,
40                 output_wh=None,
41                 transform=None,
42                 seq_transform=None,
43                 is_train=True,
44                 fp1_fpath=None,
45     ):
46         self._dataset      = dataset
47         self._transform     = transform
48         self._seq_transform = seq_transform
49         self._input_wh      = input_wh
50         self._output_wh     = input_wh if output_wh is None else output_wh
51
52         self._fp1_fpath = fp1_fpath
53
54         self._hm_generator = select_heatmap_generator(cfg['dataloader']['heatmap'])
55
56         self._is_train     = is_train
57
58         if is_train:
59             self._color_jitter_p      =
60             self._color_jitter_brightness =
61             self._color_jitter_contrast =
62             self._color_jitter_saturation =
63             self._color_jitter_hue      =
64         else:
65             self._color_jitter_p      =
66             self._color_jitter_brightness =
67             self._color_jitter_contrast =
68             self._color_jitter_saturation =
69             self._color_jitter_hue      =
70
71         self._rgb_diff = cfg['model']['rgb_diff']
72         self._frames_in = cfg['model']['frames_in']
73         self._out_scales = cfg['model']['out_scales']
74         if len(self._out_scales)!=1:
75             assert 0
76         if self._rgb_diff and self._frames_in!=2:
77             raise ValueError('rgb_diff=True supported only with frames_in=2')
78
79     def __len__(self):
80         return len(self._dataset)
81
82     def __getitem__(self, index):
83
84         fp1_im_list = []
85         if self._fp1_fpath is not None:
86             if osp.exists(self._fp1_fpath):
87                 with open(self._fp1_fpath, 'r') as f:
88                     data = f.read().split('\n')

```

```

89         fp1_im_list = data[:-1]
90
91         img_paths = self._dataset[index]['frames']
92         annos      = self._dataset[index]['annos']
93
94         img_paths_out = []
95         for anno in annos:
96             img_paths_out.append( anno['frame_path'])
97
98         input_w, input_h  = self._input_wh
99         output_w, output_h = self._output_wh
100
101         trans_input      = None
102         trans_input_inv  = None
103         trans_outputs    = defaultdict(list)
104         trans_outputs_inv = defaultdict(list)
105
106         apply_color_jitter = True
107         if random.uniform(0,1) >= self._color_jitter_p:
108             apply_color_jitter = False
109         else:
110             brightness_factor, contrast_factor, saturation_factor, hue_factor =
get_color_jitter_factors(self._color_jitter_brightness, self._color_jitter_contrast,
self._color_jitter_saturation, self._color_jitter_hue)
111
112         imgs, xys, visis = [], [], []
113         hms = defaultdict(list)
114
115         for idx, img_path in enumerate(img_paths):
116             img = read_image(img_path)
117
118             if trans_input is None:
119                 trans_input = get_transform(np.asarray(img), self._input_wh)
120                 out_w, out_h = self._output_wh
121                 for scale in self._out_scales:
122                     trans_outputs[scale] = get_transform(np.asarray(img), (out_w,
out_h))
123
124                     out_w = out_w // 2
125                     out_h = out_h // 2
126
127                 if not self._is_train:
128                     trans_input_inv = get_transform(np.asarray(img), self._input_wh,
inv=1)
129
130                     out_w, out_h      = self._output_wh
131                     for scale in self._out_scales:
132                         trans_outputs_inv[scale] = get_transform(np.asarray(img),
(out_w, out_h), inv=1)
133
134                         out_w = out_w // 2
135                         out_h = out_h // 2
136
137             img = Image.fromarray( cv2.warpAffine(np.array(img), trans_input,
self._input_wh, flags=cv2.INTER_LINEAR) )
138             imgs.append(img)
139
140             # hms (targets)
141             for idx, (img_path, anno) in enumerate(zip(img_paths, annos)):
142                 binary = True
143                 if img_path in fp1_im_list:
144                     binary = False
145
146                 px, py = anno['center'].xy
147                 visi  = anno['center'].is_visible

```

```

148         out_w, out_h = self._output_wh
149         for scale in self._out_scales:
150             if visi:
151                 ct = affine_transform(np.array([px,py]), trans_outputs[scale])
152                 ct_int = ct.astype(np.int32)
153                 hm = self._hm_generator((out_w,out_h), ct_int, binary=binary)
154             else:
155                 hm = self._hm_generator((out_w,out_h), (-1.,-1.))
156
157             hm = np.expand_dims(hm, axis=0)
158             hms[scale].append(hm)
159             out_w = out_w // 2
160             out_h = out_h // 2
161
162         imgs_t = []
163         hms_t = defaultdict(list)
164         for img in imgs:
165             # color jitter (data augmentation)
166             if apply_color_jitter:
167                 img = TF.adjust_brightness(img, brightness_factor)
168                 img = TF.adjust_contrast(img, contrast_factor)
169                 img = TF.adjust_saturation(img, saturation_factor)
170                 img = TF.adjust_hue(img, hue_factor)
171             img_t = self._transform(img)
172             imgs_t.append(img_t)
173         for scale in self._out_scales:
174             for hm in hms[scale]:
175                 hm_t = torch.tensor(hm)
176                 hms_t[scale].append(hm_t)
177
178         if self._rgb_diff:
179             if len(imgs_t)!=2:
180                 raise ValueError('assume 2 images are input but {}
given'.format(len(imgs_t)))
181             imgs_t[0] = torch.abs(imgs_t[1] - imgs_t[0])
182
183         imgs_t = torch.cat(imgs_t, dim=0)
184         for scale in self._out_scales:
185             hms_t[scale] = torch.cat(hms_t[scale], dim=0)
186         if self._seq_transform is not None:
187             imgs_t, hms_t = self._seq_transform(imgs_t, hms_t)
188         xys = torch.tensor(xys)
189         visis = torch.tensor(visis)
190         if self._is_train:
191             return imgs_t, hms_t
192         else:
193             return imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths_out
194
195
196

```

17. assistant (2026-07-06T20:41:34.609Z)

Now I have the critical detail. The `trans` is `trans\_outputs\_inv` (the inverse affine that maps output/heatmap coords back to original pixels). Note the dataset returns a tuple where `batch[2]` is `trans\_outputs\_inv`. Let me read the image utils (affine transform), model builder, and configs.

18. user (2026-07-06T20:41:37.672Z)

```

1      # -----
2      # Copyright (c) Microsoft
3      # Licensed under the MIT License.
4      # Written by Bin Xiao (Bin.Xiao@microsoft.com)
5      # Modified by Xingyi Zhou

```

```

6      # -----
7
8      from __future__ import absolute_import
9      from __future__ import division
10     from __future__ import print_function
11
12     from typing import List, Tuple
13     import numpy as np
14     import cv2
15     import random
16
17     def flip(img):
18         return img[:, :, ::-1].copy()
19
20     def transform_preds(coords, center, scale, output_size):
21         target_coords = np.zeros(coords.shape)
22         trans = get_affine_transform(center, scale, 0, output_size, inv=1)
23         for p in range(coords.shape[0]):
24             target_coords[p, 0:2] = affine_transform(coords[p, 0:2], trans)
25         return target_coords
26
27
28     def get_affine_transform(center,
29                             scale,
30                             rot,
31                             output_size,
32                             shift=np.array([0, 0], dtype=np.float32),
33                             inv=0):
34         if not isinstance(scale, np.ndarray) and not isinstance(scale, list):
35             scale = np.array([scale, scale], dtype=np.float32)
36
37         scale_tmp = scale
38         src_w = scale_tmp[0]
39         dst_w = output_size[0]
40         dst_h = output_size[1]
41
42         rot_rad = np.pi * rot / 180
43         src_dir = get_dir([0, src_w * -0.5], rot_rad)
44         dst_dir = np.array([0, dst_w * -0.5], np.float32)
45
46         src = np.zeros((3, 2), dtype=np.float32)
47         dst = np.zeros((3, 2), dtype=np.float32)
48         src[0, :] = center + scale_tmp * shift
49         src[1, :] = center + src_dir + scale_tmp * shift
50         dst[0, :] = [dst_w * 0.5, dst_h * 0.5]
51         dst[1, :] = np.array([dst_w * 0.5, dst_h * 0.5], np.float32) + dst_dir
52
53         src[2:, :] = get_3rd_point(src[0, :], src[1, :])
54         dst[2:, :] = get_3rd_point(dst[0, :], dst[1, :])
55
56         if inv:
57             trans = cv2.getAffineTransform(np.float32(dst), np.float32(src))
58         else:
59             trans = cv2.getAffineTransform(np.float32(src), np.float32(dst))
60
61         return trans
62
63
64     """
65     def affine_transform(pt, t):
66         print(pt, pt[0], pt[1])
67         new_pt = np.array([pt[0], pt[1], 1.], dtype=np.float32).T
68         print(new_pt)
69         new_pt = np.dot(t, new_pt)
70         print(new_pt)

```

```

71     return new_pt[:2]
72     """
73 def affine_transform(pt, t):
74     #print(pt,pt[0],pt[1])
75     #print(isinstance(pt, Tuple[float, float]))
76     #print(isinstance(pt, tuple))
77     #print(isinstance(pt, np.ndarray))
78     if not ( isinstance(pt, np.ndarray) and pt.shape==(2,)):
79         raise RuntimeError('pt must be np.ndarray and pt.shape=(2,)\')
80     #print(t, t.shape)
81     if not ( isinstance(t, np.ndarray) and t.shape==(2,3) ):
82         raise RuntimeError('t must be np.ndarray and t.shape=(2,3)\')
83     new_pt = np.array([pt[0], pt[1], 1.], dtype=np.float32).T
84     #print(new_pt)
85     new_pt = np.dot(t, new_pt)
86     #print(new_pt)
87     return new_pt[:2]
88
89 def get_3rd_point(a, b):
90     direct = a - b
91     return b + np.array([-direct[1], direct[0]], dtype=np.float32)
92
93
94 def get_dir(src_point, rot_rad):
95     sn, cs = np.sin(rot_rad), np.cos(rot_rad)
96
97     src_result = [0, 0]
98     src_result[0] = src_point[0] * cs - src_point[1] * sn
99     src_result[1] = src_point[0] * sn + src_point[1] * cs
100
101     return src_result
102
103
104 def crop(img, center, scale, output_size, rot=0):
105     trans = get_affine_transform(center, scale, rot, output_size)
106
107     dst_img = cv2.warpAffine(img,
108                             trans,
109                             (int(output_size[0]), int(output_size[1])),
110                             flags=cv2.INTER_LINEAR)
111
112     return dst_img
113
114
115 def gaussian_radius(det_size, min_overlap=0.7):
116     height, width = det_size
117
118     a1 = 1
119     b1 = (height + width)
120     c1 = width * height * (1 - min_overlap) / (1 + min_overlap)
121     sq1 = np.sqrt(b1 ** 2 - 4 * a1 * c1)
122     r1 = (b1 + sq1) / 2
123
124     a2 = 4
125     b2 = 2 * (height + width)
126     c2 = (1 - min_overlap) * width * height
127     sq2 = np.sqrt(b2 ** 2 - 4 * a2 * c2)
128     r2 = (b2 + sq2) / 2
129
130     a3 = 4 * min_overlap
131     b3 = -2 * min_overlap * (height + width)
132     c3 = (min_overlap - 1) * width * height
133     sq3 = np.sqrt(b3 ** 2 - 4 * a3 * c3)
134     r3 = (b3 + sq3) / 2
135     return min(r1, r2, r3)

```

```

136
137
138 def gaussian2D(shape, sigma=1):
139     m, n = [(ss - 1.) / 2. for ss in shape]
140     y, x = np.ogrid[-m:m+1, -n:n+1]
141
142     h = np.exp(-(x * x + y * y) / (2 * sigma * sigma))
143     h[h < np.finfo(h.dtype).eps * h.max()] = 0
144     return h
145
146 def draw_umich_gaussian(heatmap, center, radius, k=1):
147     diameter = 2 * radius + 1
148     gaussian = gaussian2D((diameter, diameter), sigma=diameter / 6)
149
150     x, y = int(center[0]), int(center[1])
151
152     height, width = heatmap.shape[0:2]
153
154     left, right = min(x, radius), min(width - x, radius + 1)
155     top, bottom = min(y, radius), min(height - y, radius + 1)
156
157     masked_heatmap = heatmap[y - top:y + bottom, x - left:x + right]
158     masked_gaussian = gaussian[radius - top:radius + bottom, radius - left:radius + right]
159     if min(masked_gaussian.shape) > 0 and min(masked_heatmap.shape) > 0: # TODO debug
160         np.maximum(masked_heatmap, masked_gaussian * k, out=masked_heatmap)
161     return heatmap
162
163 def draw_dense_reg(regmap, heatmap, center, value, radius, is_offset=False):
164     diameter = 2 * radius + 1
165     gaussian = gaussian2D((diameter, diameter), sigma=diameter / 6)
166     value = np.array(value, dtype=np.float32).reshape(-1, 1)
167     dim = value.shape[0]
168     reg = np.ones((dim, diameter*2+1, diameter*2+1), dtype=np.float32) * value
169     if is_offset and dim == 2:
170         delta = np.arange(diameter*2+1) - radius
171         reg[0] = reg[0] - delta.reshape(1, -1)
172         reg[1] = reg[1] - delta.reshape(-1, 1)
173
174     x, y = int(center[0]), int(center[1])
175
176     height, width = heatmap.shape[0:2]
177
178     left, right = min(x, radius), min(width - x, radius + 1)
179     top, bottom = min(y, radius), min(height - y, radius + 1)
180
181     masked_heatmap = heatmap[y - top:y + bottom, x - left:x + right]
182     masked_regmap = regmap[:, y - top:y + bottom, x - left:x + right]
183     masked_gaussian = gaussian[radius - top:radius + bottom,
184                               radius - left:radius + right]
185     masked_reg = reg[:, radius - top:radius + bottom,
186                      radius - left:radius + right]
187     if min(masked_gaussian.shape) > 0 and min(masked_heatmap.shape) > 0: # TODO debug
188         idx = (masked_gaussian >= masked_heatmap).reshape(
189             1, masked_gaussian.shape[0], masked_gaussian.shape[1])
190         masked_regmap = (1-idx) * masked_regmap + idx * masked_reg
191     regmap[:, y - top:y + bottom, x - left:x + right] = masked_regmap
192     return regmap
193
194
195 def draw_msra_gaussian(heatmap, center, sigma):
196     tmp_size = sigma * 3
197     mu_x = int(center[0] + 0.5)
198     mu_y = int(center[1] + 0.5)
199     w, h = heatmap.shape[0], heatmap.shape[1]
200     ul = [int(mu_x - tmp_size), int(mu_y - tmp_size)]

```

```

201     br = [int(mu_x + tmp_size + 1), int(mu_y + tmp_size + 1)]
202     if ul[0] >= h or ul[1] >= w or br[0] < 0 or br[1] < 0:
203         return heatmap
204     size = 2 * tmp_size + 1
205     x = np.arange(0, size, 1, np.float32)
206     y = x[:, np.newaxis]
207     x0 = y0 = size // 2
208     g = np.exp(- ((x - x0) ** 2 + (y - y0) ** 2) / (2 * sigma ** 2))
209     g_x = max(0, -ul[0]), min(br[0], h) - ul[0]
210     g_y = max(0, -ul[1]), min(br[1], w) - ul[1]
211     img_x = max(0, ul[0]), min(br[0], h)
212     img_y = max(0, ul[1]), min(br[1], w)
213     heatmap[img_y[0]:img_y[1], img_x[0]:img_x[1]] = np.maximum(
214         heatmap[img_y[0]:img_y[1], img_x[0]:img_x[1]],
215         g[g_y[0]:g_y[1], g_x[0]:g_x[1]])
216     return heatmap
217
218 def grayscale(image):
219     return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
220
221 def lighting_(data_rng, image, alphastd, eigval, eigvec):
222     alpha = data_rng.normal(scale=alphastd, size=(3, ))
223     image += np.dot(eigvec, eigval * alpha)
224
225 def blend_(alpha, image1, image2):
226     image1 *= alpha
227     image2 *= (1 - alpha)
228     image1 += image2
229
230 def saturation_(data_rng, image, gs, gs_mean, var):
231     alpha = 1. + data_rng.uniform(low=-var, high=var)
232     blend_(alpha, image, gs[:, :, None])
233
234 def brightness_(data_rng, image, gs, gs_mean, var):
235     alpha = 1. + data_rng.uniform(low=-var, high=var)
236     image *= alpha
237
238 def contrast_(data_rng, image, gs, gs_mean, var):
239     alpha = 1. + data_rng.uniform(low=-var, high=var)
240     blend_(alpha, image, gs_mean)
241
242 def color_aug(data_rng, image, eig_val, eig_vec):
243     functions = [brightness_, contrast_, saturation_]
244     random.shuffle(functions)
245
246     gs = grayscale(image)
247     gs_mean = gs.mean()
248     for f in functions:
249         f(data_rng, image, gs, gs_mean, 0.4)
250     lighting_(data_rng, image, 0.1, eig_val, eig_vec)
251

```

19. user (2026-07-06T20:41:38.698Z)

```

1     from torchvision.transforms import *
2     from PIL import Image
3     import random
4     import math
5
6     class ResizeWithEqualScale(object):
7         """
8         Resize an image with equal scale as the original image.
9         Args:
10             height (int): resized height.
11             width (int): resized width.

```

```

12         interpolation: interpolation manner.
13         fill_color (tuple): color for padding.
14     """
15     def __init__(self, height, width, interpolation=Image.BILINEAR, fill_color=(0,0,0)):
16         self.height = height
17         self.width = width
18         self.interpolation = interpolation
19         self.fill_color = fill_color
20
21     def __call__(self, img):
22         width, height = img.size
23         if self.height / self.width >= height / width:
24             height = int(self.width * (height / width))
25             width = self.width
26         else:
27             width = int(self.height * (width / height))
28             height = self.height
29
30         resized_img = img.resize((width, height), self.interpolation)
31         new_img = Image.new('RGB', (self.width, self.height), self.fill_color)
32         new_img.paste(resized_img, (int((self.width - width) / 2), int((self.height -
height) / 2)))
33
34         return new_img
35
36
37     class RandomCropping(object):
38         """
39         With a probability, first increase image size to (1 + 1/8), and then perform random
crop.
40
41         Args:
42             p (float): probability of performing this transformation. Default: 0.5.
43         """
44         def __init__(self, p=0.5, interpolation=Image.BILINEAR):
45             self.p = p
46             self.interpolation = interpolation
47
48         def __call__(self, img):
49             """
50             Args:
51                 img (PIL Image): Image to be cropped.
52             Returns:
53                 PIL Image: Cropped image.
54             """
55             width, height = img.size
56             if random.uniform(0, 1) >= self.p:
57                 return img
58
59             new_width, new_height = int(round(width * 1.125)), int(round(height * 1.125))
60             resized_img = img.resize((new_width, new_height), self.interpolation)
61             x_maxrange = new_width - width
62             y_maxrange = new_height - height
63             x1 = int(round(random.uniform(0, x_maxrange)))
64             y1 = int(round(random.uniform(0, y_maxrange)))
65             cropped_img = resized_img.crop((x1, y1, x1 + width, y1 + height))
66
67             return cropped_img
68
69     class RandomErasing(object):
70         """
71         Randomly selects a rectangle region in an image and erases its pixels.
72         Reference:
73             Zhong et al. Random Erasing Data Augmentation. arxiv: 1708.04896, 2017.
74         Args:

```

```

75         probability: The probability that the Random Erasing operation will be
performed.
76         sl: Minimum proportion of erased area against input image.
77         sh: Maximum proportion of erased area against input image.
78         r1: Minimum aspect ratio of erased area.
79         mean: Erasing value.
80     """
81
82     def __init__(self, probability = 0.5, sl = 0.02, sh = 0.4, r1 = 0.3, mean=[0.4914,
0.4822, 0.4465]):
83         self.probability = probability
84         self.mean = mean
85         self.sl = sl
86         self.sh = sh
87         self.r1 = r1
88
89     def __call__(self, img):
90
91         if random.uniform(0, 1) >= self.probability:
92             return img
93
94         for attempt in range(100):
95             area = img.size()[1] * img.size()[2]
96
97             target_area = random.uniform(self.sl, self.sh) * area
98             aspect_ratio = random.uniform(self.r1, 1/self.r1)
99
100            h = int(round(math.sqrt(target_area * aspect_ratio)))
101            w = int(round(math.sqrt(target_area / aspect_ratio)))
102
103            if w < img.size()[2] and h < img.size()[1]:
104                x1 = random.randint(0, img.size()[1] - h)
105                y1 = random.randint(0, img.size()[2] - w)
106                if img.size()[0] == 3:
107                    img[0, x1:x1+h, y1:y1+w] = self.mean[0]
108                    img[1, x1:x1+h, y1:y1+w] = self.mean[1]
109                    img[2, x1:x1+h, y1:y1+w] = self.mean[2]
110                else:
111                    img[0, x1:x1+h, y1:y1+w] = self.mean[0]
112            return img
113
114        return img
115
116

```

20. user (2026-07-06T20:41:40.225Z)

```

1     from .UNET2d import TrackNetV2
2     from .resUNET2d import ChangsTrackNet
3     from .monotrack import MonoTrack
4     from .hrnet import HRNet
5     from .deepball import DeepBall
6     from .ballseg import BallSeg
7
8     __factory = {
9         'tracknetv2': TrackNetV2,
10        'monotrack': MonoTrack,
11        'restracknetv2': ChangsTrackNet,
12        'hrnet': HRNet,
13        'deepball': DeepBall,
14        'ballseg': BallSeg
15    }
16
17    def build_model(cfg):
18        model_name = cfg['model']['name']

```

```

19         if not model_name in __factory.keys():
20             raise KeyError('invalid model: {}'.format(model_name ))
21         if model_name=='tracknetv2' or model_name=='resunet2d' or model_name=='monotrack':
22             frames_in  = cfg['model']['frames_in']
23             frames_out  = cfg['model']['frames_out']
24             bilinear    = cfg['model']['bilinear']
25             halve_channel = cfg['model']['halve_channel']
26             model       = __factory[model_name]( frames_in*3, frames_out, bilinear=bilinear,
halve_channel=halve_channel)
27         elif model_name=='higher_hrnet' or model_name=='cls_hrnet' or model_name=='hrnet':
28             model = __factory[model_name](cfg['model'])
29         elif model_name=='restracknetv2':
30             frames_in      = cfg['model']['frames_in']
31             frames_out      = cfg['model']['frames_out']
32             halve_channel   = cfg['model']['halve_channel']
33             mode            = cfg['model']['mode']
34             neck_channels   = cfg['model']['neck_channels']
35             out_mid_channels = cfg['model']['out_mid_channels']
36             blocks         = cfg['model']['blocks']
37             channels        = cfg['model']['channels']
38             model = __factory[model_name]( frames_in*3,
39                                           frames_out,
40                                           mode=mode,
41                                           halve_channel=halve_channel,
42                                           neck_channels=neck_channels,
43                                           out_mid_channels=out_mid_channels,
44                                           blocks=blocks, channels=channels)
45         elif model_name=='deepball':
46             frames_in      = cfg['model']['frames_in']
47             frames_out      = cfg['model']['frames_out']
48             class_out       = cfg['model']['class_out']
49             block_channels  = cfg['model']['block_channels']
50             block_maxpools  = cfg['model']['block_maxpools']
51             first_conv_kernel_size = cfg['model']['first_conv_kernel_size']
52             last_conv_kernel_size  = cfg['model']['last_conv_kernel_size']
53             first_conv_stride      = cfg['model']['first_conv_stride']
54             model = __factory[model_name]( frames_in*3, frames_out*class_out,
55                                           block_channels=block_channels,
56                                           block_maxpools=block_maxpools,
57                                           first_conv_kernel_size=first_conv_kernel_size,
58                                           first_conv_stride=first_conv_stride,
59                                           last_conv_kernel_size=last_conv_kernel_size)
60         elif model_name=='ballseg':
61             frames_in      = cfg['model']['frames_in']
62             frames_out      = cfg['model']['frames_out']
63             scale_factors   = cfg['model']['scale_factors']
64             backbone        = cfg['model']['backbone']
65             model = __factory[model_name]( in_channels=frames_in*3, nclass=frames_out*1,
backbone=backbone, scale_factors=scale_factors)
66
67         return model
68
69

```

21. user (2026-07-06T20:41:40.774Z)

```

1     from .detector import TracknetV2Detector
2     from .deepball_detector import DeepBallDetector
3
4     __factory = {
5         'tracknetv2' : TracknetV2Detector,
6         'deepball': DeepBallDetector
7     }
8
9     def build_detector(cfg, model=None):

```

```

10     detector_name = cfg['detector']['name']
11     if not detector_name in __factory.keys():
12         raise KeyError('invalid detector: {}'.format(detector_name ))
13     return __factory[detector_name](cfg, model=model)
14
15

```

22. assistant (2026-07-06T20:41:45.007Z)

Now let me read the transform builder (`build\_img\_transforms`), the configs, and check how the `run\_tensor` is invoked (the `trans` passed is `trans\_outputs\_inv`, which is a dict keyed by scale). Let me read the key configs and the dataloaders `\_\_init\_\_`.

23. user (2026-07-06T20:41:47.470Z)

```

1     import os
2     import os.path as osp
3     from omegaconf import DictConfig
4     from torch.utils.data import DataLoader
5
6     import dataloaders.img_transforms as T
7     import dataloaders.seq_transforms as ST
8
9     from .dataset_loader import ImageDataset, read_image, get_transform
10    from .samplers import select_sampler, RandomSampler
11    from datasets import select_dataset
12
13    def build_img_transforms(cfg):
14        transform_train = T.Compose([
15            T.ToTensor(),
16            T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
17        ])
18        transform_test = T.Compose([
19            T.ToTensor(),
20            T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
21        ])
22        return transform_train, transform_test
23
24    def build_seq_transforms(cfg):
25        transform_train = ST.SeqTransformCompose([
26            ST.RandomHorizontalFlipping(cfg['transform']['train']['horizontal_flip']['p']),
27            ST.RandomCropping(p=cfg['transform']['train']['crop']['p'],
max_rescale=cfg['transform']['train']['crop']['max_rescale']),
28        ])
29        transform_test = None
30        return transform_train, transform_test
31
32    def build_dataloader(
33        cfg: DictConfig,
34    ):
35        dataset = select_dataset(cfg)
36
37        transform_train, transform_test = build_img_transforms(cfg)
38        seq_transform_train, seq_transform_test = build_seq_transforms(cfg)
39
40        input_wh = (cfg['model']['inp_width'], cfg['model']['inp_height'])
41        output_wh = (cfg['model']['out_width'], cfg['model']['out_height'])
42
43        train_sampler, test_sampler, train_clip_samplers, test_clip_samplers =
select_sampler(cfg['dataloader']['sampler'], dataset)
44
45        model_name = cfg['model']['name']
46
47        fp1_fname = cfg['runner']['fp1_filename']
48        if fp1_fname is not None:

```

```

49         fp1_fpath = osp.join(cfg['output_dir'], fp1_fname)
50     else:
51         fp1_fpath = None
52
53     train_clip_datasets = {}
54     test_clip_datasets = {}
55     if model_name in ['tracknetv2', 'hrnet', 'monotrack', 'restracknetv2', 'deepball',
56 'ballseg']:
57         train_dataset = ImageDataset(cfg,
58                                     dataset=dataset.train,
59                                     input_wh=input_wh,
60                                     output_wh=output_wh,
61                                     transform=transform_train,
62                                     seq_transform=seq_transform_train,
63                                     fp1_fpath=fp1_fpath,
64                                     )
65         test_dataset = ImageDataset(cfg,
66                                    dataset=dataset.test,
67                                    input_wh=input_wh,
68                                    output_wh=output_wh,
69                                    transform=transform_test,
70                                    seq_transform=seq_transform_test,
71                                    is_train=False,
72                                    )
73         for key, clip in dataset.train_clips.items():
74             clip_dataset = ImageDataset(cfg,
75                                         dataset=clip,
76                                         input_wh=input_wh,
77                                         output_wh=output_wh,
78                                         transform=transform_test,
79                                         is_train=False,
80                                         )
81             train_clip_datasets[key] = clip_dataset
82
83         for key, clip in dataset.test_clips.items():
84             clip_dataset = ImageDataset(cfg,
85                                         dataset=clip,
86                                         input_wh=input_wh,
87                                         output_wh=output_wh,
88                                         transform=transform_test,
89                                         is_train=False,
90                                         )
91             test_clip_datasets[key] = clip_dataset
92     else:
93         raise ValueError('unknwon model_name : {}'.format(model_name))
94
95
96     train_loader = DataLoader(dataset=train_dataset,
97                              batch_sampler=train_sampler,
98                              num_workers=cfg['data_loader']['train_num_workers'],
99                              pin_memory=False)
100    test_loader = DataLoader(dataset=test_dataset,
101                             batch_sampler=test_sampler,
102                             num_workers=cfg['data_loader']['test_num_workers'],
103                             pin_memory=False)
104
105    train_clip_loaders_and_gts = {}
106    for key, clip_dataset in train_clip_datasets.items():
107        clip_loader = DataLoader(dataset=clip_dataset,
108                                batch_sampler=train_clip_samplers[key],
109                                num_workers=cfg['data_loader']['inference_video_num_workers'],
110                                pin_memory=False)
111        train_clip_loaders_and_gts[key] = {'clip_loader': clip_loader, 'clip_gt':

```

```

dataset.train_clip_gts[key]]
112
113     test_clip_loaders_and_gts = {}
114     for key, clip_dataset in test_clip_datasets.items():
115         clip_loader = DataLoader(dataset=clip_dataset,
116                                 batch_sampler=test_clip_samplers[key],
117
num_workers=cfg['data_loader']['inference_video_num_workers'],
118                                pin_memory=False)
119     test_clip_loaders_and_gts[key] = {'clip_loader': clip_loader, 'clip_gt':
dataset.test_clip_gts[key]]
120
121     return train_loader, test_loader, train_clip_loaders_and_gts,
test_clip_loaders_and_gts
122
123

```

24. user (2026-07-06T20:41:48.987Z)

```

1     name: hrnet
2     frames_in: 3
3     frames_out: 3
4     inp_height: 288
5     inp_width: 512
6     out_height: 288
7     out_width: 512
8     rgb_diff: False
9     out_scales: [0]
10    MODEL:
11        EXTRA:
12            FINAL_CONV_KERNEL: 1
13            PRETRAINED_LAYERS: ['*']
14            STEM:
15                INPLANES: 64
16                STRIDES: [1,1]
17            STAGE1:
18                NUM_MODULES: 1
19                NUM_BRANCHES: 1
20                BLOCK: BOTTLENECK
21                NUM_BLOCKS: [1]
22                NUM_CHANNELS: [32]
23                FUSE_METHOD: SUM
24            STAGE2:
25                NUM_MODULES: 1
26                NUM_BRANCHES: 2
27                BLOCK: BASIC
28                NUM_BLOCKS: [2,2]
29                NUM_CHANNELS: [16,32]
30                FUSE_METHOD: SUM
31            STAGE3:
32                NUM_MODULES: 1
33                NUM_BRANCHES: 3
34                BLOCK: BASIC
35                NUM_BLOCKS: [2,2,2]
36                NUM_CHANNELS: [16,32,64]
37                FUSE_METHOD: SUM
38            STAGE4:
39                NUM_MODULES: 1
40                NUM_BRANCHES: 4
41                BLOCK: BASIC
42                NUM_BLOCKS: [2,2,2,2]
43                NUM_CHANNELS: [16,32,64,128]
44                FUSE_METHOD: SUM
45        DECONV:
46            NUM_DECONVS: 0

```

```
47         KERNEL_SIZE: []
48         NUM_BASIC_BLOCKS: 2
49         INIT_WEIGHTS: True
50
51
```

25. user (2026-07-06T20:41:49.991Z)

```
1     name: badminton
2     root_dir: /root/datasets/badminton
3     video_dirname: video
4     frame_dirname: frame
5     csv_dirname: csv
6     train:
7         matches : [ 'match1',
8                     'match2',
9                     'match3',
10                    'match4',
11                    'match5',
12                    'match6',
13                    'match7',
14                    'match8',
15                    'match9',
16                    'match10',
17                    'match11',
18                    'match12',
19                    'match13',
20                    'match14',
21                    'match15',
22                    'match16',
23                    'match17',
24                    'match18',
25                    'match19',
26                    'match20',
27                    'match21',
28                    'match22',
29                    'match23',
30                    'match24',
31                    'match25',
32                    'match26']
33     num_clip_ratio: 1.0
34     test:
35         matches: ['test_match1',
36                 'test_match2',
37                 'test_match3']
38     num_clip_ratio: 1.0
39
```

26. user (2026-07-06T20:41:50.995Z)

```
1     defaults:
2         - _self_
3     name: tracknetv2
4     model_path: # None
5     step: 3
6     postprocessor:
7         name: tracknetv2
8         score_threshold: 0.5
9         scales: [0]
10        blob_det_method: concomp
11        use_hm_weight: True
12
13
```

27. user (2026-07-06T20:41:52.863Z)

```
1      name: online
2      max_disp: 300
3
4
```

28. user (2026-07-06T20:41:53.261Z)

```
1      defaults:
2          - _self_
3          - runner: eval
4          - dataset: badminton
5          - model: wasb
6          - dataloader: default
7          - detector: tracknetv2
8          - transform: default
9          - tracker: online
10     hydra:
11         run:
12             dir: /root/outputs/${hydra.job.name}/${now:%Y-%m-%d_%H-%M-%S}
13     output_dir:
14     seed: 1234
15
16
```

29. assistant (2026-07-06T20:41:56.601Z)

Now I have the full picture. Let me read the remaining key pieces: the dataloader default config (heatmap sigmas), the HRNet model signature, and `utils/\_\_init\_\_` (for `Center`, `read\_image`, `\_nms`, `\_top1`).

30. user (2026-07-06T20:41:59.662Z)

```
1      train: False
2      test: False
3      train_clip: False
4      test_clip: True
5      #inference_video: True
6      sampler:
7          name: random
8          train_batch_size: 8
9          train_shuffle_batch: True
10         train_drop_last: True
11         test_batch_size: 8
12         test_shuffle_batch: False
13         test_drop_last: False
14         inference_video_batch_size: 8
15         inference_video_shuffle_batch: False
16         inference_video_drop_last: False
17     train_num_workers: 8
18     test_num_workers: 8
19     inference_video_num_workers: 8
20     heatmap:
21         name: binary_fixed_size
22         sigmas: [2.5]
23         mags: [1.0]
24         min_value: 0.6
25
26
```

31. user (2026-07-06T20:42:00.690Z)

```
1      name: eval
```

```
2     device: cuda
3     gpus: [0,1,2,3]
4     vis_result: False
5     vis_hm: False
6     vis_traj: False
7     #best_model_name: best_model.pth.tar
8     #model_dir:
9     model_path:
10    split: test
11    fp1_filename: # dummy
12    eval:
13        score_threshold: 0.5
14        dist_threshold: 4
15
16
```

32. user (2026-07-06T20:42:02.197Z)

```
1     train:
2         color_jitter:
3             p: 0.
4             brightness: 0.
5             contrast: 0.
6             saturation: 0.
7             hue: 0.
8         horizontal_flip:
9             p: 0.
10        crop:
11            p: 0.
12            max_rescale: 0.125
13    test:
14        color_jitter:
15            p: 0.
16            brightness: 0.
17            contrast: 0.
18            saturation: 0.
19            hue: 0.
20        horizontal_flip:
21            p: 0.
22        crop:
23            p: 0.
24            max_rescale: 0.125
25
```

33. user (2026-07-06T20:42:03.322Z)

```
1     # -----
2     # Copyright (c) Microsoft
3     # Licensed under the MIT License.
4     # Written by Bin Xiao (leoxiaobin@gmail.com)
5     # Modified by Bowen Cheng (bcheng9@illinois.edu)
6     # -----
7
8     from __future__ import absolute_import
9     from __future__ import division
10    from __future__ import print_function
11
12    from __future__ import absolute_import
13    from __future__ import division
14    from __future__ import print_function
15
16    import os
17    import logging
18
19    import torch
```

```

20     import torch.nn as nn
21
22
23     BN_MOMENTUM = 0.1
24     logger = logging.getLogger(__name__)
25
26
27     def conv3x3(in_planes, out_planes, stride=1):
28         """3x3 convolution with padding"""
29         return nn.Conv2d(in_planes, out_planes, kernel_size=3, stride=stride,
30                         padding=1, bias=False)
31
32
33     class BasicBlock(nn.Module):
34         expansion = 1
35
36         def __init__(self, inplanes, planes, stride=1, downsample=None):
37             super(BasicBlock, self).__init__()
38             self.conv1 = conv3x3(inplanes, planes, stride)
39             self.bn1 = nn.BatchNorm2d(planes, momentum=BN_MOMENTUM)
40             self.relu = nn.ReLU(inplace=True)
41             self.conv2 = conv3x3(planes, planes)
42             self.bn2 = nn.BatchNorm2d(planes, momentum=BN_MOMENTUM)
43             self.downsample = downsample
44             self.stride = stride
45
46         def forward(self, x):
47             residual = x
48
49             out = self.conv1(x)
50             out = self.bn1(out)
51             out = self.relu(out)
52
53             out = self.conv2(out)
54             out = self.bn2(out)
55
56             if self.downsample is not None:
57                 residual = self.downsample(x)
58
59             out += residual
60             out = self.relu(out)

```

34. assistant (2026-07-06T20:42:06.293Z)

Now let me check the HRNet forward output format (what `preds[scale]` looks like), and confirm `read\_image`/`Center`/`\_nms`/`\_top1` locations in utils.

35. user (2026-07-06T20:42:11.056Z)

No matches found

[Showing results with pagination = offset: 300]

36. user (2026-07-06T20:42:11.685Z)

```

1     from .utils import save_checkpoint, set_seed, mkdir_if_missing, count_params,
AverageMeter, list2txt, read_image, compute_l2_dist_mat
2     from .heatmap import gen_heatmap, gen_binary_map
3     from .dataclasses import Center
4     from .file import load_csv_tennis
5     from .refine_gt import refine_gt_clip_tennis
6     from .vis import draw_frame, gen_video
7     from .evaluator import Evaluator
8

```

37. assistant (2026-07-06T20:42:14.121Z)

Let me see the HRNet forward method and its output structure to confirm how `preds[scale]` is indexed.

38. user (2026-07-06T20:42:15.872Z)

```

29:     return nn.Conv2d(in_planes, out_planes, kernel_size=3, stride=stride,
46:     def forward(self, x):
62:         return out
83:     def forward(self, x):
103:         return out
158:         layers.append(block(self.num_inchannels[branch_index],
163:             layers.append(block(self.num_inchannels[branch_index],
166:         return nn.Sequential(*layers)
172:         branches.append(
175:     return nn.ModuleList(branches)
179:     return None
188:         fuse_layer.append(nn.Sequential(
198:         fuse_layer.append(None)
204:             conv3x3s.append(nn.Sequential(
211:             conv3x3s.append(nn.Sequential(
217:         fuse_layer.append(nn.Sequential(*conv3x3s))
218:         fuse_layers.append(nn.ModuleList(fuse_layer))
220:     return nn.ModuleList(fuse_layers)
223:     return self.num_inchannels
225:     def forward(self, x):
227:         return [self.branches[0](x[0])]
241:         x_fuse.append(self.relu(y))
243:     return x_fuse
260:     self._out_scales = cfg['out_scales']
311:     self.final_layers = self._make_final_layers(cfg, pre_stage_channels)
324:     return deconv_kernel, padding, output_padding
326:     def _make_final_layers(self, cfg, channels):
329:         for scale in self._out_scales:
330:             layers.append( nn.Conv2d(in_channels=channels[scale],
out_channels=self._frames_out, kernel_size=kernel_size) )
331:             return nn.ModuleList(layers)
343:             layers.append(nn.Sequential(
355:                 deconv_layers.append(nn.Sequential(*layers))
358:             return nn.ModuleList(deconv_layers)
369:                 transition_layers.append(nn.Sequential(
380:                 transition_layers.append(None)
387:                 conv3x3s.append(nn.Sequential(
392:                 transition_layers.append(nn.Sequential(*conv3x3s))
394:             return nn.ModuleList(transition_layers)
406:             layers.append(block(inplanes, planes, stride, downsample))
409:             layers.append(block(inplanes, planes))
411:             return nn.Sequential(*layers)
430:             modules.append(
441:             return nn.Sequential(*modules), num_inchannels
443:     def forward(self, x):
455:         x_list.append(self.transition1[i](x))
457:         x_list.append(x)
458:         y_list = self.stage2(x_list)
463:         x_list.append(self.transition2[i](y_list[-1]))
465:         x_list.append(y_list[i])
466:         y_list = self.stage3(x_list)
471:         x_list.append(self.transition3[i](y_list[-1]))
473:         x_list.append(y_list[i])
474:         y_list = self.stage4(x_list)
477:         for scale in self._out_scales:
478:             x = y_list[scale]

```

```

481:         y = self.final_layers[scale](x)
483:     return y_out

```

39. user (2026-07-06T20:42:23.653Z)

```

443     def forward(self, x):
444         x = self.conv1(x)
445         x = self.bn1(x)
446         x = self.relu(x)
447         x = self.conv2(x)
448         x = self.bn2(x)
449         x = self.relu(x)
450         x = self.layer1(x)
451
452         x_list = []
453         for i in range(self.stage2_cfg['NUM_BRANCHES']):
454             if self.transition1[i] is not None:
455                 x_list.append(self.transition1[i](x))
456             else:
457                 x_list.append(x)
458         y_list = self.stage2(x_list)
459
460         x_list = []
461         for i in range(self.stage3_cfg['NUM_BRANCHES']):
462             if self.transition2[i] is not None:
463                 x_list.append(self.transition2[i](y_list[-1]))
464             else:
465                 x_list.append(y_list[i])
466         y_list = self.stage3(x_list)
467
468         x_list = []
469         for i in range(self.stage4_cfg['NUM_BRANCHES']):
470             if self.transition3[i] is not None:
471                 x_list.append(self.transition3[i](y_list[-1]))
472             else:
473                 x_list.append(y_list[i])
474         y_list = self.stage4(x_list)
475
476         y_out = {}
477         for scale in self._out_scales:
478             x = y_list[scale]
479             for i in range(self.num_deconvs):
480                 x = self.deconv_layers[i][scale](x)
481             y = self.final_layers[scale](x)
482             y_out[scale] = y
483         return y_out
484
485     def init_weights(self, pretrained=''):
486         logger.info('=> init weights from normal distribution')
487         for m in self.modules():

```

40. assistant (2026-07-06T20:42:29.216Z)

Now I understand the full model output: `forward` returns a `dict {scale: tensor[B, frames\_out=3, H, W]}`. The postprocessor iterates `preds[scale]` for `scale in [0]`, calls `.sigmoid()`, and decodes per (batch i, sequence-frame j) ? where `j` indexes the 3 output channels (frames_out=3, one per frame in the window). So `run\_tensor` returns `results[bid][eid]` where `eid` is the per-window frame index (0,1,2).

Let me verify the HRNet constructor signature and check the `\_nms`/`\_top1` utils briefly, plus confirm how `trans` (the `trans\_outputs\_inv` dict) is consumed by `run\_tensor` / postprocessor ? it indexes `affine\_mats[scale]`.

41. user (2026-07-06T20:42:31.032Z)

```

245
246     blocks_dict = {
247         'BASIC': BasicBlock,
248         'BOTTLENECK': Bottleneck
249     }
250
251
252     # https://github.com/HRNet/HRNet-Image-
Classification/blob/master/lib/models/cls_hrnet.py
253     class HRNet(nn.Module):
254
255         def __init__(self, cfg, **kwargs):
256             super(HRNet, self).__init__()
257
258             self._frames_in = cfg['frames_in']
259             self._frames_out = cfg['frames_out']
260             self._out_scales = cfg['out_scales']
261             self._stem_strides = cfg['MODEL']['EXTRA']['STEM']['STRIDES']
262             self._stem_inplanes = cfg['MODEL']['EXTRA']['STEM']['INPLANES']
263
264             self.conv1 = nn.Conv2d(3*self._frames_in, self._stem_inplanes, kernel_size=3,
stride=self._stem_strides[0], padding=1, bias=False)
265             self.bn1 = nn.BatchNorm2d(self._stem_inplanes, momentum=BN_MOMENTUM)
266             self.conv2 = nn.Conv2d(self._stem_inplanes, self._stem_inplanes, kernel_size=3,
stride=self._stem_strides[1], padding=1, bias=False)
267             self.bn2 = nn.BatchNorm2d(self._stem_inplanes, momentum=BN_MOMENTUM)
268             self.relu = nn.ReLU(inplace=True)
269
270             self.stage1_cfg = cfg['MODEL']['EXTRA']['STAGE1']
271             num_channels = self.stage1_cfg['NUM_CHANNELS'][0]
272             block = blocks_dict[self.stage1_cfg['BLOCK']]
273             num_blocks = self.stage1_cfg['NUM_BLOCKS'][0]
274             self.layer1 = self._make_layer(block, self._stem_inplanes, num_channels,
num_blocks)
275             stage1_out_channel = block.expansion*num_channels
276
277             self.stage2_cfg = cfg['MODEL']['EXTRA']['STAGE2']
278             num_channels = self.stage2_cfg['NUM_CHANNELS']
279             block = blocks_dict[self.stage2_cfg['BLOCK']]
280             num_channels = [
281                 num_channels[i] * block.expansion for i in range(len(num_channels))]
282             self.transition1 = self._make_transition_layer(
283                 [stage1_out_channel], num_channels)
284             self.stage2, pre_stage_channels = self._make_stage(
285                 self.stage2_cfg, num_channels)
286
287             self.stage3_cfg = cfg['MODEL']['EXTRA']['STAGE3']
288             num_channels = self.stage3_cfg['NUM_CHANNELS']
289             block = blocks_dict[self.stage3_cfg['BLOCK']]
290             num_channels = [
291                 num_channels[i] * block.expansion for i in range(len(num_channels))]
292             self.transition2 = self._make_transition_layer(
293                 pre_stage_channels, num_channels)
294             self.stage3, pre_stage_channels = self._make_stage(
295                 self.stage3_cfg, num_channels)
296
297             self.stage4_cfg = cfg['MODEL']['EXTRA']['STAGE4']
298             num_channels = self.stage4_cfg['NUM_CHANNELS']
299             block = blocks_dict[self.stage4_cfg['BLOCK']]
300             num_channels = [
301                 num_channels[i] * block.expansion for i in range(len(num_channels))]
302             self.transition3 = self._make_transition_layer(
303                 pre_stage_channels, num_channels)
304             self.stage4, pre_stage_channels = self._make_stage(self.stage4_cfg,
num_channels, multi_scale_output=True)

```

```

305
306         self.num_deconvs          = cfg['MODEL']['EXTRA']['DECONV']['NUM_DECONVS']
307         self.deconv_config         = cfg['MODEL']['EXTRA']['DECONV']
308         self.pretrained_layers     = cfg['MODEL']['EXTRA']['PRETRAINED_LAYERS']
309
310         self.deconv_layers = self._make_deconv_layers(cfg, pre_stage_channels[0])
311         self.final_layers  = self._make_final_layers(cfg, pre_stage_channels)
312
313     def _get_deconv_cfg(self, deconv_kernel):
314         if deconv_kernel == 4:

```

42. assistant (2026-07-06T20:42:45.046Z)

I now have everything I need. Let me confirm the key facts before writing:

```

- **HRNet** built via `HRNet(cfg['model'])` where `cfg['model']` is the wasb.yaml dict (has
`frames_in`, `frames_out`, `out_scales`, `MODEL.EXTRA.*`). The spike used
`HRNet(OmegaConf.create(cfg))`.
- **Model output**: `dict {0: tensor[B,3,288,512]}`.
- **Postprocessor** (`TracknetV2Postprocessor`): needs `cfg['detector']['postprocessor']`
(score_threshold=0.5, scales=[0], blob_det_method=concomp, use_hm_weight=True),
`cfg['model']['name']`, `cfg['data_loader']['heatmap']['sigmas']=[2.5]`. Its `.run(preds,
affine_mats)` takes `preds` dict `{scale: tensor}` and `affine_mats` dict `{scale:
tensor[B,2,3]}`.
- **`run_tensor(imgs, affine_mats)`**: `imgs` = `[B, 9, 288, 512]`, `affine_mats` = the
`trans_outputs_inv` dict `{0: tensor[B,2,3]}`. Returns `results[bid][eid]` = list of `{xy,
score, scale}`.
- **`trans`** built via `get_transform(img, (out_w,out_h), inv=1)` = inverse affine from heatmap
(512x288) coords -> original pixels. Since out_wh == inp_wh == (512,288), the heatmap coords ARE
at 512x288.
- **Tracker** (`OnlineTracker`): needs `cfg['tracker']['max_disp']=300`. `refresh()` then
`update(frame_dets)` per frame id in order.
- **det accumulation** (from reference): per window, for each `eid` (0,1,2), map back to real
frame id `p = img_paths[eid][batch]`, `fid = _frame_id(p)`, accumulate
`det_by_fid[fid].extend(dets)`. Since window i covers frames [i, i+1, i+2], each frame appears
in up to 3 windows -> candidates accumulate.

```

Let me verify the `\_nms`/`\_top1` import in postprocessor doesn't break (they're imported but the concomp path doesn't use them), and quickly check the `datasets` import chain doesn't get pulled by importing `build\_detector`.

43. user (2026-07-06T20:42:48.927Z)

```

1:import os.path as osp
2:import logging
3:from hydra.core.hydra_config import HydraConfig
4:import numpy as np
5:from PIL import Image
6:import cv2
7:import torch
8:from torch import nn
10:from models import build_model
11:from dataloaders import read_image, get_transform, build_img_transforms
12:from utils.image import get_affine_transform, affine_transform
14:from .deepball_postprocessor import DeepBallPostprocessor

```

44. user (2026-07-06T20:42:50.370Z)

```

87:def _nms(heat, kernel=3):
95:def _top1(scores):

```

45. assistant (2026-07-06T20:43:00.734Z)

Importing `detectors` pulls in `dataloaders` which imports `datasets` (`from datasets import

select_dataset`). That could pull heavy deps, but the reference `wasb_infer.py` imports `build_detector` fine, so it works in the env. However, to follow the prompt's guidance ("Prefer building the model directly, avoid Hydra runtime, but USE WASB's own postprocessor + tracker classes"), I'll import `TracknetV2Postprocessor` and `OnlineTracker` directly and build HRNet directly ? avoiding the `DataParallel`/`build_detector` path (which asserts `device=='cuda'` and hard-wires gpus) while still using WASB's exact decode + tracker classes.

I have all I need. Let me verify the eval harness / baseline CSV format expectation by checking how the reference writes rows (already seen: `Frame,Visibility,X,Y` with `1` if visi else `0`, `x`, `y`).

I now have complete understanding. Writing the script.

46. user (2026-07-06T20:43:01.733Z)

Chapter marked: "Write exp_t2_full.py". Continue your current work.

47. assistant (2026-07-06T20:45:08.006Z)

I have everything needed. Here is the complete script and notes.

```
```python
#!/usr/bin/env python3
"""exp_t2_full.py ? FULL WASB shuttle-detection inference under torch 2.x with two
interchangeable front-ends (CPU cv2 vs GPU-resident PyNvVideoCodec).
```

Runs on a Linux L4 VM (NOT the authoring box). It builds WASB's HRNet directly (no Hydra runtime, no DataParallel) but reuses WASB's OWN postprocessor (`TracknetV2Postprocessor`) and tracker (`OnlineTracker`) classes so the produced trajectory matches the reference `wasb\_infer.py` exactly.

For each front-end it streams the clip (sliding 3-frame window, O(window) memory), runs heatmap -> postprocess -> per-frame detection accumulation -> OnlineTracker, and writes `~/t2/{frontend}\_traj.csv` with header `Frame,Visibility,X,Y` in the ORIGINAL 1920x1080 pixel space (via the inverse affine `trans`, exactly as the reference builds it).

Layout assumed on the VM:

```
~/t2/WASB-SBDT (repo; ~/t2/WASB-SBDT/src added to sys.path)
~/t2/wasb.pth.tar (weights: torch.load(...)["model_state_dict"])
~/t2/g1.mp4 (golden clip, 1080p ~59.94fps, ~10448 frames)
~/t2/g1_traj_baseline.csv (cached torch-1.11 baseline Frame,Visibility,X,Y)
```

Output: `~/t2/cpu\_traj.csv`, `~/t2/gpu\_traj.csv`, plus a parity summary on stdout.  
"""

```
import csv
import functools
import os
import os.path as osp
import sys
import time
from collections import defaultdict
```

```
import numpy as np
```

```
print = functools.partial(print, flush=True) # unbuffered
```

----- #

## Paths / constants

----- #

```
T2 = osp.expanduser("~/t2")
```

```

REPO_SRC = osp.join(T2, "WASB-SBDT", "src")
WEIGHTS = osp.join(T2, "wasb.pth.tar")
VIDEO = osp.join(T2, "g1.mp4")
BASELINE_CSV = osp.join(T2, "g1_traj_baseline.csv")

```

## Make WASB's own modules importable (postprocessor + tracker + image utils + hrnet)

```
sys.path.insert(0, REPO_SRC)
```

## Model / preprocessing constants (from configs/model/wasb.yaml + build\_img\_transfo

```

FRAMES_IN = 3 # 3 RGB frames -> 9 input channels
FRAMES_OUT = 3 # model emits 3 heatmap channels (one per window frame)
INP_W, INP_H = 512, 288
OUT_W, OUT_H = 512, 288 # out == inp; heatmap coords already at 512x288
SCALE = 0 # out_scales == [0]
IMAGENET_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
IMAGENET_STD = np.array([0.229, 0.224, 0.225], dtype=np.float32)

```

## Optional cap for smoke tests: set env T2\_LIMIT to a frame count. 0 / unset => full clip.

```
LIMIT = int(os.environ.get("T2_LIMIT", "0") or "0")
```

----- #

**Build the WASB cfg dict the postprocessor + tracker + HRNet expect.**

**We hand-build the exact sub-tree Hydra would compose (eval.yaml defaults) so we can instantiate WASB's OWN classes WITHOUT running Hydra.**

----- #

```

def build_cfg():
 from omegaconf import OmegaConf

 cfg = {
 "model": {
 "name": "hrnet",
 "frames_in": FRAMES_IN,
 "frames_out": FRAMES_OUT,
 "inp_height": INP_H,
 "inp_width": INP_W,
 "out_height": OUT_H,
 "out_width": OUT_W,
 "rgb_diff": False,
 "out_scales": [0],
 # HRNet architecture (configs/model/wasb.yaml, MODEL.EXTRA.*)
 "MODEL": {
 "EXTRA": {
 "FINAL_CONV_KERNEL": 1,
 "PRETRAINED_LAYERS": [""],
 "STEM": {"INPLANES": 64, "STRIDES": [1, 1]},
 "STAGE1": {
 "NUM_MODULES": 1, "NUM_BRANCHES": 1, "BLOCK": "BOTTLENECK",
 "NUM_BLOCKS": [1], "NUM_CHANNELS": [32], "FUSE_METHOD": "SUM",
 },
 "STAGE2": {
 "NUM_MODULES": 1, "NUM_BRANCHES": 2, "BLOCK": "BASIC",
 "NUM_BLOCKS": [2, 2], "NUM_CHANNELS": [16, 32], "FUSE_METHOD": "SUM",
 },
 "STAGE3": {
 "NUM_MODULES": 1, "NUM_BRANCHES": 3, "BLOCK": "BASIC",
 "NUM_BLOCKS": [2, 2, 2], "NUM_CHANNELS": [16, 32, 64], "FUSE_METHOD":
SUM",
 },
 },
 },
 },
 }

```

```

 "STAGE4": {
 "NUM_MODULES": 1, "NUM_BRANCHES": 4, "BLOCK": "BASIC",
 "NUM_BLOCKS": [2, 2, 2, 2], "NUM_CHANNELS": [16, 32, 64, 128],
"FUSE_METHOD": "SUM",
 },
 "DECONV": {"NUM_DECONVS": 0, "KERNEL_SIZE": [], "NUM_BASIC_BLOCKS": 2},
 },
 "INIT_WEIGHTS": True,
},
},
configs/detector/tracknetv2.yaml
"detector": {
 "name": "tracknetv2",
 "model_path": WEIGHTS,
 "step": 3,
 "postprocessor": {
 "name": "tracknetv2",
 "score_threshold": 0.5,
 "scales": [0],
 "blob_det_method": "concomp",
 "use_hm_weight": True,
 },
},
},
configs/dataloader/default.yaml (postprocessor reads heatmap.sigmas)
"dataloader": {"heatmap": {"name": "binary_fixed_size", "sigmas": [2.5],
 "mags": [1.0], "min_value": 0.6}},
configs/tracker/online.yaml
"tracker": {"name": "online", "max_disp": 300},
configs/runner/eval.yaml (postprocessor doesn't need it here, but harmless)
"runner": {"device": "cuda", "gpus": [0]},
}
return OmegaConf.create(cfg)

```

----- #

**Affine transforms (replicate dataloaders.dataset\_loader.get\_transform exactly).**  
**forward: original(HxW) -> 512x288 input (used to warp each frame the CPU way)**  
**inverse: 512x288 heatmap -> original px (the `trans` fed to run\_tensor)**  
**Both are keyed off the ORIGINAL frame size, so they are constant for the clip.**

----- #

```

def build_transforms(orig_h, orig_w):
 from utils.image import get_affine_transform

 c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
 s = max(orig_h, orig_w) * 1.0
 # forward input warp (original -> 512x288), inv=0
 trans_input = get_affine_transform(c, s, 0, [INP_W, INP_H], inv=0)
 # inverse output map (512x288 heatmap -> original px), inv=1 == reference's `trans`
 trans_out_inv = get_affine_transform(c, s, 0, [OUT_W, OUT_H], inv=1)
 return trans_input.astype(np.float32), trans_out_inv.astype(np.float32)

```

----- #

**The detector: HRNet forward + WASB's OWN postprocessor.**  
**Mirrors detectors/detector.py::run\_tensor but built directly (no DataParallel,**  
**no cuda-only assert) so it runs cleanly under torch 2.x on the L4.**

----- #

```

class Detector:
 def __init__(self, cfg, device):
 import torch
 from models.hrnet import HRNet
 from detectors.postprocessor import TracknetV2Postprocessor

 self.device = device
 self.model = HRNet(cfg["model"]).to(device)
 ckpt = torch.load(WEIGHTS, map_location=device, weights_only=False)
 state = ckpt["model_state_dict"]
 # weights may be saved from a DataParallel model ("module." prefix) ? strip it.
 if any(k.startswith("module.") for k in state):
 state = {k[len("module.")]: v for k, v in state.items()}
 missing, unexpected = self.model.load_state_dict(state, strict=False)
 if missing or unexpected:
 print(f"[detector] load_state_dict: {len(missing)} missing, "
 f"{len(unexpected)} unexpected keys")
 self.model.eval()
 self.postprocessor = TracknetV2Postprocessor(cfg)

 def run_tensor(self, imgs, affine_mats):
 """imgs: [B,9,288,512] float on-device; affine_mats: {scale: Tensor[B,2,3]}.
 Returns results[bid][eid] -> list of {'xy', 'score', 'scale'} (eid in 0..2)."""
 import torch

 with torch.no_grad():
 imgs = imgs.to(self.device, non_blocking=True)
 preds = self.model(imgs) # {0: [B,3,288,512]}
 pp = self.postprocessor.run(preds, affine_mats)
 results = {}
 for bid in sorted(pp.keys()):
 results[bid] = {}
 for eid in sorted(pp[bid].keys()):
 dets = []
 for sc in sorted(pp[bid][eid].keys()):
 scores = pp[bid][eid][sc]["scores"]
 xys = pp[bid][eid][sc]["xys"]
 for xy, score in zip(xys, scores):
 dets.append({"xy": xy, "score": score, "scale": sc})
 results[bid][eid] = dets
 return results

```

----- #

## Window tensor assembly.

#

**CPU front-end: cv2 decode (BGR) -> RGB -> cv2.warpAffine to 512x288 -> /255 -> ImageNet-normalize. This is byte-equivalent to WASB's disk path (read\_image PIL RGB -> warpAffine -> ToTensor(/255) -> Normalize).**

#

**GPU front-end: PyNvVideoCodec decodes RGB HWC uint8 on GPU -> dlpack tensor -> permute CHW -> F.interpolate bilinear to 512x288 -> /255 -> ImageNet-normalize. (F.interpolate is NOT the same math as the affine warp; for a 16:9 1080p clip into a 16:9 512x288 target the affine is a pure isotropic resize with no pad, so bilinear-resize is the close GPU analogue. Any residual diff surfaces in the parity summary ? that IS the experiment.)**

----- #

```
def normalize_chw_np(chw_rgb_float):
 """chw_rgb_float: [3,H,W] float32 in [0,1] -> ImageNet-normalized in place-ish."""
 out = np.empty_like(chw_rgb_float)
 for c in range(3):
 out[c] = (chw_rgb_float[c] - IMAGENET_MEAN[c]) / IMAGENET_STD[c]
 return out
```

## ---- CPU front-end ----- #

```
class CpuFrontend:
 """Forward-only cv2 decoder yielding normalized 512x288 CHW float tensors (CPU),
 plus per-frame original-resolution size (for the affine)."""

 name = "cpu"

 def __init__(self, video_path):
 import cv2

 self.cv2 = cv2
 self.cap = cv2.VideoCapture(video_path)
 if not self.cap.isOpened():
 raise RuntimeError(f"cannot open {video_path}")
 self.count = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
 self.orig_w = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH) or 0)
 self.orig_h = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT) or 0)
 self.trans_input, self.trans_out_inv = build_transforms(self.orig_h, self.orig_w)
 self._pos = 0

 def read_chw(self, index):
 import numpy as np

 assert index == self._pos, f"cpu decode is forward-only ({index} != {self._pos})"
 ok, frame_bgr = self.cap.read()
 if not ok:
 raise RuntimeError(f"failed to decode frame {index}")
 self._pos += 1
 rgb = self.cv2.cvtColor(frame_bgr, self.cv2.COLOR_BGR2RGB)
 warped = self.cv2.warpAffine(rgb, self.trans_input, (INP_W, INP_H),
 flags=self.cv2.INTER_LINEAR) # HWC uint8
 chw = warped.transpose(2, 0, 1).astype(np.float32) / 255.0 # ToTensor
 return normalize_chw_np(chw)

 def to_tensor(self, chw_np):
 import torch

 return torch.from_numpy(np.ascontiguousarray(chw_np))

 def stack_window(self, chw_list):
 import torch

 return torch.cat([self.to_tensor(c) for c in chw_list], dim=0) # [9,288,512]

 def release(self):
 self.cap.release()
```

## ---- GPU front-end ----- #

```
class GpuFrontend:
 """PyNvVideoCodec SimpleDecoder -> GPU-resident RGB -> F.interpolate resize +
 GPU ImageNet-normalize. Windows are assembled on-GPU. Forward-only random access
 into SimpleDecoder (indexable)."""

 name = "gpu"
```

```

def __init__(self, video_path):
 import torch
 import PyNvVideoCodec as nvc

 self.torch = torch
 self.nvc = nvc
 self.gpu = 0
 dev = torch.device("cuda", self.gpu)
 self.device = dev
 # Try the RGB output path; fall back defensively if it errors.
 self.rgb_native = True
 try:
 self.dec = nvc.SimpleDecoder(
 video_path, gpu_id=self.gpu,
 output_color_type=nvc.OutputColorType.RGB,
)
 except Exception as e:
 # noqa: BLE001 ? surface + fall back
 print(f"[gpu] SimpleDecoder RGB path failed ({e!r}); "
 f"falling back to default output + manual convert")
 self.rgb_native = False
 self.dec = nvc.SimpleDecoder(video_path, gpu_id=self.gpu)
 self.count = len(self.dec)
 # Probe first frame to learn original H/W and dlpack layout.
 probe = self._decode_hwc_uint8(0)
 self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
 self.trans_input, self.trans_out_inv = build_transforms(self.orig_h, self.orig_w)
 self.mean = torch.tensor(IMAGENET_MEAN, device=dev).view(3, 1, 1)
 self.std = torch.tensor(IMAGENET_STD, device=dev).view(3, 1, 1)

def _decode_hwc_uint8(self, index):
 """Return a GPU torch tensor [H,W,3] uint8 (RGB) for frame `index`."""
 torch = self.torch
 frame = self.dec[index]
 t = torch.from_dlpack(frame)
 # Layout heuristics: SimpleDecoder RGB yields HWC (H,W,3); some builds yield
 # CHW (3,H,W) or packed. Normalize to HWC uint8.
 if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
 t = t.permute(1, 2, 0).contiguous()
 if not self.rgb_native:
 # Default decode is typically NV12/planar; if we got 3 channels treat as
 # BGR-ish and swap; otherwise raise (unsupported without a color convert).
 if t.dim() == 3 and t.shape[-1] == 3:
 t = t[..., [2, 1, 0]].contiguous()
 else:
 raise RuntimeError(
 f"gpu fallback got non-3ch frame shape {tuple(t.shape)}; "
 f"a full NV12->RGB convert is needed on this build")
 return t

def read_chw(self, index):
 """[3,288,512] float32 on GPU, ImageNet-normalized."""
 import torch.nn.functional as F

 hwc = self._decode_hwc_uint8(index)
 chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
 resized = F.interpolate(chw, size=(INP_H, INP_W),
 mode="bilinear", align_corners=False)
 return (resized - self.mean) / self.std

def stack_window(self, chw_list):
 return self.torch.cat(chw_list, dim=0)

def release(self):
 self.dec = None

```

----- #

**Core: stream the clip through a front-end, run detector + tracker, write CSV.**  
**Sliding 3-frame window == the reference's build\_windows; per-window we run the detector once and accumulate per-frame candidates by REAL frame id, exactly as wasb\_infer.py does (det\_by\_fid[fid].extend(dets)). Then OnlineTracker resolves.**

----- #

```
def run(frontend_cls, out_csv, cfg, device):
 import torch

 fe = frontend_cls(VIDEO)
 total = fe.count if not LIMIT else min(fe.count, LIMIT)
 print(f"[{fe.name}] frames={fe.count} (processing {total}), "
 f"orig={fe.orig_w}x{fe.orig_h}")
 if total < FRAMES_IN:
 raise RuntimeError(f"need >= {FRAMES_IN} frames, have {total}")

 detector = Detector(cfg, device)

 # affine_mats is the inverse-output transform, per-batch, keyed by scale ? the
 # `trans` the reference feeds run_tensor. It's identical for every window (the
 # clip resolution is constant), so build a batch-of-1 tensor and reuse it.
 trans_np = fe.trans_out_inv # [2,3] float32
 is_gpu = fe.name == "gpu"
 trans_dev = device if is_gpu else torch.device("cpu")
 trans_1 = torch.from_numpy(np.ascontiguousarray(trans_np)).to(trans_dev) # [2,3]

 # Sliding buffer of the last FRAMES_IN normalized frames (CHW), forward-only.
 from trackers.online import OnlineTracker
 tracker = OnlineTracker(cfg)
 tracker.refresh() # reset stateful tracker

 det_by_fid = defaultdict(list)
 window = [] # list of (fid, chw)
 t0 = time.time()
 n_windows = 0

 for idx in range(total):
 chw = fe.read_chw(idx)
 window.append((idx, chw))
 if len(window) < FRAMES_IN:
 continue
 if len(window) > FRAMES_IN:
 window.pop(0)

 fids = [w[0] for w in window]
 imgs = fe.stack_window([w[1] for w in window]).unsqueeze(0) # [1,9,288,512]
 affine_mats = {SCALE: trans_1.unsqueeze(0)} # {0: [1,2,3]}
 results = detector.run_tensor(imgs, affine_mats) # results[0][eid]

 for eid in sorted(results[0].keys()):
 fid = fids[eid]
 det_by_fid[fid].extend(results[0][eid])
 n_windows += 1

 if n_windows % 500 == 0:
 el = time.time() - t0
 fps = (n_windows) / el if el > 0 else 0.0
 print(f"[{fe.name}] windows={n_windows}/{total - FRAMES_IN + 1} "
 f"| {fps:.1f} win/s | elapsed {el:.0f}s")

 fe.release()
```

```

el = time.time() - t0
proc_fps = total / el if el > 0 else 0.0
print(f"[{fe.name}] detection done: {total} frames, {n_windows} windows in "
 f"{el:.0f}s ({proc_fps:.1f} frames/s)")

---- tracker pass: one update() per frame id in order -----
rows = []
for fid in sorted(det_by_fid.keys()):
 # tracker expects xy as a numpy array (it does np.linalg.norm on it).
 frame_dets = [
 {"xy": np.asarray(d["xy"], dtype=float),
 "score": float(d["score"]), "scale": int(d["scale"])}
 for d in det_by_fid[fid]
]
 r = tracker.update(frame_dets)
 rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

_write_csv(out_csv, rows)
visible = sum(1 for _, v, *_ in rows if v)
print(f"[{fe.name}] wrote {len(rows)} rows -> {out_csv} "
 f"(visible {visible}/{len(rows)})")
return out_csv

def _write_csv(out_csv, rows):
 os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
 tmp = out_csv + ".tmp"
 with open(tmp, "w", newline="") as f:
 w = csv.writer(f)
 w.writerow(["Frame", "Visibility", "X", "Y"])
 for fid, vis, x, y in rows:
 w.writerow([fid, vis, x, y])
 os.replace(tmp, out_csv)

```

----- #

## Parity summary

----- #

```

def _load_traj(path):
 """path -> {frame: (vis, x, y)}."""
 out = {}
 if not osp.exists(path):
 return out
 with open(path, newline="") as f:
 r = csv.DictReader(f)
 for row in r:
 try:
 fid = int(float(row["Frame"]))
 vis = int(float(row["Visibility"]))
 x = float(row["X"])
 y = float(row["Y"])
 except (KeyError, ValueError):
 continue
 out[fid] = (vis, x, y)
 return out

def _compare(name_a, a, name_b, b):
 if not a or not b:
 print(f" {name_a} vs {name_b}: one side empty (a={len(a)}, b={len(b)}) ? skip")
 return
 common = sorted(set(a) & set(b))
 both_vis_dists = []

```

```

vis_agree = 0
for fid in common:
 va, xa, ya = a[fid]
 vb, xb, yb = b[fid]
 if va == vb:
 vis_agree += 1
 if va == 1 and vb == 1:
 both_vis_dists.append(float(np.hypot(xa - xb, ya - yb)))
vis_rate = vis_agree / len(common) if common else 0.0
if both_vis_dists:
 d = np.array(both_vis_dists)
 med = float(np.median(d))
 p95 = float(np.percentile(d, 95))
 print(f" {name_a} vs {name_b}: common={len(common)} "
 f"both-visible={len(both_vis_dists)} "
 f"| median={med:.3f}px p95={p95:.3f}px "
 f"| visibility-agreement={vis_rate*100:.2f}%")
else:
 print(f" {name_a} vs {name_b}: common={len(common)} "
 f"both-visible=0 | visibility-agreement={vis_rate*100:.2f}%")

```

```

def parity_summary(cpu_csv, gpu_csv):
 print("\n==== PARITY SUMMARY =====")
 cpu = _load_traj(cpu_csv)
 gpu = _load_traj(gpu_csv)
 base = _load_traj(BASELINE_CSV)
 print(f"loaded: cpu={len(cpu)} gpu={len(gpu)} baseline={len(base)}")
 _compare("cpu", cpu, "gpu", gpu)
 _compare("cpu", cpu, "baseline", base)
 _compare("gpu", gpu, "baseline", base)
 print("=====\n")

```

----- #

## Main

----- #

```

def main():
 import torch

 print(f"torch {torch.__version__} | cuda_available={torch.cuda.is_available()}")
 if not torch.cuda.is_available():
 print("WARNING: CUDA not available; detector will run on CPU (slow).")
 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
 cfg = build_cfg()

 cpu_csv = osp.join(T2, "cpu_traj.csv")
 gpu_csv = osp.join(T2, "gpu_traj.csv")

 # Each front-end wrapped so one failure doesn't lose the other.
 try:
 print("\n----- FRONT-END: cpu (cv2 decode) -----")
 run(CpuFrontend, cpu_csv, cfg, device)
 except Exception as e:
 # noqa: BLE001
 import traceback
 print(f"[cpu] FAILED: {e!r}")
 traceback.print_exc()

 try:
 print("\n----- FRONT-END: gpu (PyNvVideoCodec) -----")
 run(GpuFrontend, gpu_csv, cfg, device)
 except Exception as e:
 # noqa: BLE001
 import traceback

```

```

 print(f"[gpu] FAILED: {e!r}")
 traceback.print_exc()

 parity_summary(cpu_csv, gpu_csv)
 print("done.")

if __name__ == "__main__":
 main()
...

```

## Notes

**\*\*WASB classes imported (all from `~/t2/WASB-SBDT/src`, added to `sys.path`):\*\***

- Detector model: `models.hrnet.HRNet` ? built directly as `HRNet(cfg["model"])` (the wasb.yaml sub-tree wrapped in `OmegaConf.create`), weights via `torch.load(..., weights\_only=False)["model\_state\_dict"]` with a defensive `module.` prefix strip. This avoids `detectors.build\_detector`, which force-asserts `device=='cuda'` + wraps in `nn.DataParallel(gpus)` and pulls the whole `dataloaders`?`datasets` import chain.
- Postprocessor: `detectors.postprocessor.TracknetV2Postprocessor` ? used verbatim (`run(preds, affine\_mats)`, sigmoid + `concomp` blob decode + `affine\_transform` back to original px). The decode is NOT reimplemented.
- Tracker: `trackers.online.OnlineTracker` ? used verbatim, `refresh()` before each front-end run, one `update(frame\_dets)` per frame id in ascending order. `xy` is passed as a numpy array (the tracker does `np.linalg.norm` on it).
- Affine: `utils.image.get\_affine\_transform` ? used directly.

**\*\*How `trans` is built (matches `dataloaders.dataset\_loader.get\_transform`):\*\*** `c = [W/2, H/2]`, `s = max(H,W)`, then `get\_affine\_transform(c, s, 0, [512,288], inv=1)` gives the inverse map from 512x288 heatmap coords back to original 1920x1080 pixels. This is exactly the `trans\_outputs\_inv[0]` the reference returns as `batch[2]` and feeds to `run\_tensor`. Because the clip resolution is constant, this 2x3 matrix is identical for every window, so it's built once and broadcast as `{0: trans[None]}` (batch-of-1). The forward input warp (`inv=0`) is used by the CPU front-end to reproduce WASB's `cv2.warpAffine(rgb, trans\_input, (512,288))`.

**\*\*Detection accumulation (mirrors `wasb\_infer.py`):\*\*** the model emits 3 heatmap channels per 3-frame window (`frames\_out=3`); `run\_tensor` returns `results[bid][eid]` for `eid in {0,1,2}`. For window starting at frame `i`, `eid` maps to real frame id `i+eid`, and candidates are `det\_by\_fid[fid].extend(...)` ? so each frame accrues candidates from up to 3 overlapping windows before the tracker resolves. Preprocessing constants (512x288, /255, ImageNet mean/std, sigmoid, score\_threshold=0.5, concomp, use\_hm\_weight=True, sigmas=[2.5], max\_disp=300) are taken from the configs, not guessed.

**\*\*Front-end difference that the experiment measures:\*\*** the CPU path uses WASB's exact `cv2.warpAffine` (affine resize), while the GPU path uses `F.interpolate(bilinear)`. For a 16:9 1080p clip into a 16:9 512x288 target the affine is a pure isotropic downscale with no padding, so bilinear-resize is the correct GPU analogue ? but the two interpolators are not bit-identical, and that residual is precisely what the parity summary surfaces (median / p95 pixel distance on both-visible frames, plus visibility-agreement rate, cpu-vs-gpu and each-vs-baseline).

**\*\*Likely to need a tweak on first run (VM-specific):\*\***

1. **\*\*PyNvVideoCodec dlpack layout.\*\*** `SimpleDecoder[i].\_\_dlpack\_\_` shape/channel order (HWC vs CHW, RGB vs planar/NV12) varies across PyNvVideoCodec 2.x builds. `\_decode\_hwc\_uint8` normalizes HWC/CHW and best-effort handles the non-RGB fallback, but if the fallback yields NV12/planar it raises ? a real NV12?RGB conversion (or the `output\_color\_type=RGB` path succeeding) is required. The RGB path is attempted first and wrapped in try/except with a printed error.
2. **\*\*`np.Inf` in `trackers/online.py`.** WASB's `OnlineTracker.\_select\_best` uses `np.Inf`, removed in NumPy 2.0. If the L4 has NumPy ?2.0 this raises `AttributeError`; either pin NumPy <2 in the env or add `np.Inf = np.inf` shim at the top of the script (not added by default to keep the tracker source untouched ? flag it).
3. **\*\*Frame-count mismatch.** cv2's `CAP\_PROP\_FRAME\_COUNT` (CPU) and `len(SimpleDecoder)` (GPU) can disagree by a few frames on VFR/odd containers; the parity summary compares on the intersection of frame ids, so a small mismatch is tolerated but worth watching (the baseline is

~10448 frames).

4. `weights_only=False` is required for the torch-1.11-era checkpoint under torch 2.6 (default flipped to `True` in 2.6). Kept explicit.

5. `OmegaConf` cfg access. `TracknetV2Postprocessor`/`OnlineTracker`` index cfg like a dict (`cfg['tracker']['max_disp']`); `OmegaConf.create` supports that. If the env's omegaconf is very old, swap to plain nested dicts (the cfg is already a plain dict pre-`OmegaConf.create`).

I did not run anything ? the script targets the Linux L4 VM.